



代码之外

从实现者到定义者

AI 时代研发者的价值重构

易 成 著

目 录

序	接下来怎么办	3
.....		
第 一 部分	看清 到底发生了什么	
1	软件工程史，就是一部翻译史	8
.....		
2	什么在贬值，什么在升值	19
.....		
3	35 岁线的真相	33
.....		
第 二 部分	立身 价值的三个来源	
4	价值公式：为什么同样是 AI，产出差三倍	44
.....		
5	业务理解：为什么它成了立身之本	54
.....		
6	判断力：只能靠做决策、担后果来练	66
.....		
第 三 部分	跃迁 能力怎么长出来	
7	OPT：从技能互补到判断互补	80
.....		
8	五能阶梯：能做、能写、能说、能讲、能聊	89
.....		
9	驾驭者：唯一一条年轻人不吃亏的赛道	98
.....		
第 四 部分	落地 个人与组织各自怎么办	
10	小 leader 的两难：向上，还是向下沉	107
.....		
11	不是更努力，是换维度	117
.....		
12	组织这一侧：从"管人"到"带领业务发展"	126
.....		
结 语	未来在哪	136
.....		

接下来怎么办

这两年，我被人问过很多次同一个问题。

一个刚毕业的年轻人问：现在工具这么强，我还要不要下功夫学写代码？一个做了七八年的工程师问：我这几年攒下来的东西，还有用吗？一个带小组的人问：我每天在会上说的话，好像越来越没人听了。还有人问得更直接：如果有一天我不用写代码了，我干什么？

这些问题看着不一样，其实是同一个：接下来怎么办。

我没有标准答案，也不打算给。这本书不是答案，是把这件事想清楚的一次尝试。

为什么写

因为这个问题躲不开，而市面上关于它的说法，大多不太好用。

一类说法是"工具很强，你学就对了"。这话没错，但它回答不了上面那些问题，会用工具的人现在越来越多，可焦虑的人并没有变少。

另一类说法是"别慌，历史上每次技术进步都创造了更多岗位"。这话也对，但它是给行业听的，不是给人听的。行业总量会涨，不等于你现在的岗位还在，也不等于你还干得动。

第三类说法是"淘汰你的不是 AI，是会用 AI 的人"。这话有一半是对的：会用工具确实比不会用强。可惜它停在了最浅的一层：等所有人都会用工具的时候，接下来呢？它没说。

我需要一個能落到地面上的答案：在這件事里，一個人到底失去了什麼，還剩下什麼，能往哪走。

所以有了這本書。

这不是一本什么书

先把邊界劃清楚，免得你抱著錯的期待往下讀。

不是工具書。工具每個月都在變，寫進書里的操作等你讀到可能已經過時。這本書里幾乎不教具體怎么用某個工具。

不是勵志書。我不打算說"只要努力就不會被淘汰"。這句話不對，因為有些事情個人確實改變不了——崗位總數在收縮，年齡偏見真實存在，家庭成本是硬約束。這本書里有一整節專門講這些。

也不是唱衰的書。我不認為這行要完了。相反，我認為接下來十年，是這行里個體與個體之間差距拉得最大的十年。對很多人來說，這是機會。

這本書只想做一件事：把正在發生的事講清楚，然後給出幾條能走的路。

读完你能拿到什么

三樣東西。

第一，說得出問題在哪。很多人現在的不安是模糊的——只知道不對勁，說不清哪里不對勁。這本書會把它拆開：被替代的到底是哪一部分，為什麼你變快了卻更累了，35歲那條線的真正機制是什麼。從"說不清的焦慮"到"能說出問題在哪"，這一步本身就值錢。

第二，几个能自测的模型。什么在贬值、什么在升值（第 2 章），个人价值的构成（第 4 章），业务理解的五层阶梯（第 5 章），五能阶梯（第 8 章）。你可以拿它们给自己量一量，知道现在站在哪，往哪挪半步是有效的。

第三，每章末尾能直接做的事。分"个人"和"管理者"两栏。都是小动作，不是让你在现有负担上再加码，而是在你已经在做的事里换一种做法。这一点我很在意，因为这本书的读者大多已经很累了。

怎么读

如果你只有二十分钟：看每章开头那句加粗的话，加上章末的模型卡片和行动清单。核心就都在里面了。

如果你想挑着读：应届生看 1、9、11 章；在职工程师看 1、5、8 章；带团队的技术专家看 3、10、12 章；管理者看 5、6、12 章。

如果你想从头读：大约两个半到三个小时。

另外，这本书是双线的。每章末尾的行动分两栏，一栏给个人，一栏给带团队的人。因为在我看来，个人成长最终是为了成就更好的组织：一个人变强不叫转型，一群人变强才叫。

最后一句

这本书里写的不是某个人的成功经验。我在这个行业里待了很多年，见过转型成功的人，也见过被卡住的人，我自己也在半路上。

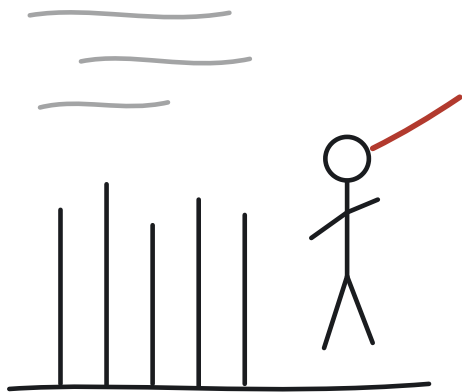
书里的故事，一部分来自真实经历，为保护当事人做了匿名和改写；一部分是为了说明某个普遍现象构造的典型情境，不指向任何具体的人。你可以当故事读，但请相信里面讲的规律。

还有一件事交代一下。这本书一开始不叫《代码之外》，叫《研发领航》。

起那个名字，是因为我当时想说的就一句话：带团队的人，第一优先级不是管人，是带领业务发展。写着写着发现，这件事不只跟带团队的人有关——不带人的工程师，一样得知道自己的方向在哪。所以书名换成了《代码之外》，副标题定成"从实现者到定义者"。

但"领航"这两个字我留着了。它不是管理，是判断方向。第 12 章会用到它，最后也还会回到它。

如果你正在问"接下来怎么办"，那我们就是同一条船上的人。往下翻吧。



PART ONE

第 一 部 分

看清

到底发生了什么

软件工程史，就是一部翻译史

AI 吃掉的不是"研发"，是"翻译"。

这一章的三件事

- 这行一直在干的一件事：把人的想法，翻成机器能执行的东西
- 提效不等于减负：你更快了，但活更多、人更少、责任更重
- 所以路只有一条——"研发"这两个字，重心得从"发"挪到"研"

—

那周五下午四点半，小周合了最后一个 PR。

那个需求原本排了两周，他这一周做完了。周一用工具把接口和数据结构先撸出来，后面几天改逻辑、补测试、修边角。周三他跟 leader 说"可能提前"，leader 回了个"👍"。

他当时挺高兴。

周会上 leader 说了句"效率挺高"，然后抬头问："那下个迭代你提前做什么？"

小周愣了一下，说还没想好。

leader 说："那把排在后面的两个需求并进来吧。"

这句话他听懂了：提前做完，换来的不是提前做完的奖励，是更多的活。

真正的变化在后面三个月。

下一个迭代，组里排的需求比上季度多了差不多一倍——理由很正当：现在做得快了嘛。同时组里走了两个人，一个转岗，一个"毕业"。走的时候没声张，工位上的东西是趁大家开会时收走的。

八个人变六个，需求翻一倍。

小周现在的状态是这样的：上午写自己的；下午 review 工具生成的那批代码——它写得快，但总有些地方不对，接口边界处理得潦草、异常分支漏掉、命名跟项目里的习惯对不上，得一条一条改；晚上还得回消息，因为需求多了，来回确认的事也跟着多了。

他跟我讲这事的时候说了句挺实在的话：

"我干得比以前快了一倍，可我比以前更累了。而且我说不清我好在哪了。"

小周不是能力差。五年后端，做过的系统不少，组里谁卡住了找他基本都能解。他的麻烦在于：他一直是個很好的执行者，而"执行得更快"这件事，正在变得既不值钱、也不轻松。

他不知道的是，他遇到的不是个人问题，是这行几十年里最大的一次结构性变化。

二 这行到底在干什么

我们换个角度想这件事。

一个做 CRM 的团队，客户说"我想知道哪些客户快跑了"。这句话离能跑的程序有多远？

中间隔着好几层。得先把它变成功能描述——流失预警，看登录频次、看工单、看续费意向；再变成数据模型、接口设计、页面结构；再变成代码；再变成能部署、能监控、能回滚的东西；最后还得变成别人能接手、能维护的东西。

每一层都是一次翻译。上一层是人的话，下一层是机器的话。

为什么必须一层层翻？因为机器只听得懂最后那一层。你没法直接告诉数据库"哪些客户快跑了"，它不知道什么叫快跑。所以很长一段时间里，"会翻译"本身就是稀缺技能——你得同时听懂上面那层人在说什么，和下面那层机器要什么。

而且每翻一层都有损耗。客户说的"快跑了"，可能指三个月没下单；产品经理理解成"活跃度下降"；开发理解成"最近登录次数少"。传三轮，意思已经不一样了。这行里大量的返工、扯皮和加班，本质上都是翻译损耗。

所以过去几十年，"翻译得快、翻得准、少出错"就是这行的核心能力。我们管它叫熟练度。

回头看看这行的历史，其实很清楚：

模 型

这行到底在干什么

年代	发生了什么	翻译成本怎么变的
早期	机器语言、汇编	人得用机器的话说话
六七十年代	高级语言	人可以用接近人的话说话了
八九十年代	框架、库、组件	不用从零翻了，有现成模块
两千年代	开源生态、云	基础设施不用自己搭，翻的部分更少了
近十年	低代码、Serverless、标准化 SaaS	相当一部分"翻"被平台接走
现在	AI	从"想法"直接到"能跑的东西"

每一代都有人说"程序员要没了"。每一代都没说对，因为翻译成本一降，软件就变便宜，需求随之暴涨，要翻的东西反而更多了。做网站的人没因为 WordPress 变少，做 App 的人没因为云变少。

同时人会往上挪一层：从写汇编挪到写 C，从自己搭服务器挪到写业务逻辑，从手搓界面挪到调组件库。每一层被工具接走了，人就站到上面一层去。

过去每一轮，这个上挪都花了差不多十年。行业有足够时间让一整代人完成迁移。

这次的麻烦在两点。

一是挪的方向变了。过去是"从更底层挪到更高层"，上面还有地方可去。这次是"从翻译本身挪出去"——目的地不是更高的抽象层，而是翻译之外的地方：定义问题、判断取舍、理解业务。这两件事的难度不一样，前者是学新工具，后者是换一种工作方式。

二是快得多。从能补全一行代码，到能独立干完一个模块，工具用了不到三年。人换一种工作方式，三年不够。

过去每一代，降的是翻译的"机械部分"——语法不用记了、环境不用配了、轮子不用造了。但从意图到结构的那一步，一直牢牢在人手里。需求怎么拆、数据怎么建模、边界怎么定，这些从来都是人干的。

这次被接走的，正是这一步。

你给 AI 一句"我想知道哪些客户快跑了"，它能给你一份像模像样的数据模型、接口定义、甚至跑起来的原型。它写得对不对是另一回事，但这一步不再必须经过人了。

这才是真正的不一样：以前是人一步步往上升，工具在下面托着；这次是工具直接爬到了人站的那一层。

三 研，和发

中文里"研发"这个词用得特别顺，顺到我们很少拆开想。

研，是研究：要造什么、为什么造、造出来解决谁的什么问题、值不值得造。发，是开发：把它做出来，做对、做快、做稳。

理想里这两个都重要。现实里呢？

大部分研发人员一天的工作是什么样的：需求文档已经写好了，原型已经画好了，接口约定已经在会上过过了。你要做的，是把它变成能跑的代码，再把它交出去。

拿一个工程师的一周来看，大概是这样：写代码和改 bug 占大头，接着是开会（过需求、对进度、评方案），然后是处理线上问题、写文档、做 code review。真正花在"这东西该不该做、有没有更好的做法"上的时间，一周里可能不到一小时，而那一小时通常还是在会上听别人说。

八成以上在"发"，"研"那部分早被别人做完了——被产品经理、被架构师、被 leader 做完了。

这种分工不是错误，它正是过去几十年效率提升的来源。翻译很贵，所以让专业的人专门翻，是最划算的安排。分工越细，翻得越快。

问题在于：当翻译不再贵，这套分工的前提就没了。而被分工框住的那部分人，从来没练过分工之外的本事。

所以过去评价一个工程师好不好，看的是发的能力：做得快不快、bug 多不多、边界处理得稳不稳、出了问题能不能救。这些都很重要，但它们全都发生在"目标已经定了"的前提下。

现在的麻烦是：发这一整层，正在被工具接管。

不是全部，也不是明天。但方向很清楚。公开过的对照实验里，用上辅助工具的那组完成任务普遍快出三成到一半，具体数字差得很远，取决于任务类型和实验怎么设计，别把某一个数当结论。近两年各家机构出的报告，指向的方向是一致的：越是结构清晰、目标明确、能验证对错的活，提升越明显。

什么样的活符合这三条？就是"发"的活。

所以那句听起来很吓人的话，得改一下说法：

不是"AI 会取代研发人员"，是"AI 会取代研发里'发'的那部分"。

而"研"那部分——判断该不该做、做哪个、做成什么样算好，不但没被取代，反而因为执行变便宜了，变得更值钱。

过去重心在"发"，未来重心在"研"。从实现者到定义者。

这句话是这本书的主心骨，后面十一章都在围着它转。

小周那句"我说不清我还需要会什么"，答案就在这儿：他需要的不是再学一门语言、多熟一个框架，是学会在目标还没定的时候就开始工作。

四 为什么更快了，却更累了

这一章很容易被读成"以后大家没事干了"。正好相反——眼下真实发生的是：提效了，但没减负。

三个机制，一个比一个实在。

第一，效率提升会被自动吸收进工作预期。

你用工具三天做完了一周的活，下一次排期就是三天。这不是谁在为难你，这是组织的常识——既然能三天做完，那三天就是合理的。

效率增益没有沉淀成你的闲暇，也没有沉淀成你的议价空间，它变成了新标准。

这件事不难理解：技术进步带来的收益归谁，取决于议价能力。当执行可替代性强的时候，议价能力就弱，增益自然归组织。这是结构，不是态度。

第二，活变多和人变少，是同时发生的。

企业拿到 AI 这张牌，第一反应不是"让大家轻松点"，而是两件事一起做：一边接更多需求、要求更快上线，一边重新计算——完成这些活，到底需要几个人。

所以"被优化"和"留下的人更累"是同一个过程的两面。走的人难受，留下的人也不轻松。

第三，工具还带来了一批新的活。

这块最容易被忽略。工具写得快，但写得不一定对。你要 review，要改它潦草的边界处理，要补它漏掉的异常分支，要把它生成的东西跟项目里既有的风格对上。

产出增加的同时，责任没减少，还多了一道质检。

以前你是生产者。现在你是生产者 + 质检员 + 最终责任人。生产那段变快了，另外两段都在变重。而且这三段里，只有生产那段能被工具分担。有个组专门统计过，review 工具产出的时间占到一个迭代的两成左右，这个活两年前是不存在的。

所以真实的状态不是"没事干"，是——活更多、人更少、责任更重、心里更没底。

这一条很重要，它直接决定了这本书的主张：如果答案是"更努力、更快、接更多活"，那你只是在这个循环里跑得更快一点而已。跑得越快，标准抬得越快。第 11 章专门讲这件事。

五 熟练度不是都跌，得拆开看

上一节说熟练度贬值，这话得说准，不然会误导。

熟练度其实是三样东西捆在一起的：

模 型

熟练度不是都跌，得拆开看

熟练度里有什么	走势	原因
手速：语法熟、API 记得住、敲得快	跌得最狠	纯记忆和机械动作，工具完胜
手感：知道哪里容易出事、这段代码改下去会牵连什么	半涨	属于预判，有价值，但工具正在学会
判断：这个场景该不该用这个框架、什么时候宁可自己写	涨	需要知道代价，需要知道业务约束

"手感"这一项值得多说一句。干了十年的人，看一眼改动就知道"这块碰不得"，这种直觉非常值钱，也是很多老工程师最后的安全垫。但它正在被侵蚀——工具读过的代码比任何人都多，它能提示的风险面在变宽。

不过有个东西它短期拿不走：这种直觉建立在"知道这个系统为什么长成这样"之上，而这其实是靠向"研"那一侧的东西。

所以准确的说法是：跌的是翻译的速度，涨的是翻译过程中做出的判断。

六 先别急着焦虑，看看几个常见的误读

写到这儿我想插一句。这一章很容易被读成"技术不重要了"，那不是我的意思，得说清楚。

误读一：那我不用学技术了？ 不是这样。技术的价值从"会做"转移到了"知道能不能做、代价多大、坑在哪"。你让 AI 写一段代码，得知道它写得好不好、有没有埋雷、上线会出什么事。什么都不懂的人，连验收都不会。真正危险的不是"技术差"，是只有技术。

误读二：那是不是都得转产品、转管理？ 也不是。"研"不是岗位，是工作方式。写代码的人照样可以做"研"——只要你开始关心"这东西为什么做、做成什么样算好、还有没有更好的解法"，你就在研了。

误读三：这次是不是狼来了？ 前面说了，历史上每代都有人说程序员要没了。这次不一样的地方在于，被接走的是"从意图到结构"那一步。但也要诚实：这个过程会是渐进的、不均匀的。有的领域快，有的慢；有的公司激进，有的保守。你不会在某天早上突然失业。更可能的是像小周那样——某天发现自己的产出翻倍了，而自己的处境一点没变好：活更多，人更少，说不清自己好在哪。

也有一句要提醒：别把疲惫当成努力的证明。现在很多人比两年前累得多，但这种累换来的东西正在变少。累是真实的，问题在于它是不是在正确的方向上。

自检三问

- 1 你这个月做的事里，有多少是"目标已经由别人定好了，你负责实现"的？
- 2 如果 AI 把实现的时间砍掉一半，多出来的时间你打算做什么？想得出来吗？
- 3 上一次有人说"这事不该这么做"、并且是你说的，是什么时候？

可以做的事

个人

- 1 下次接到需求，先别动手。花十分钟写下三个答案：为什么要做、为谁做、做完什么样算好。写不出来的，去问
- 2 挑一件你现在做的活，试着自己重新定义一遍问题，不是"怎么实现它"，是"它到底要解决什么"
- 3 别急着囤工具。工具会变，判断力不会。这周少学一个新框架，多搞懂一个你做的东西背后的业务

管理者

- 1 估算工时的时候，别再默认"实现占大头"。多出来的时间要提前规划给"研"，否则它会自动变成无效加班
- 2 看人的时候，把"做得快不快"和"问得对不对"分开看。后者现在更值钱
- 3 给团队一个明确的信号：会提"这个不该做"的人，不会被当成刺头

别的岗位呢

这章的"翻译"，不是程序员独有的。

设计师把"要高级一点"翻成版式和配色，是翻译。产品经理把老板一句话翻成原型和文档，是翻译。律师把客户的诉求翻成合同条款，财务把经营状况翻成报表口径，咨询把客户的问题翻成一份报告结构——都是翻译。

区别只在速度。代码的产物是数字化的，改一个字就能重跑，所以最先被侵入。别的岗位慢一点，但机理一样：凡是"把一种话翻成另一种话"的活，都在同一条曲线上。

什么在贬值，什么在升值

中间塌陷，两头崛起。而先塌的是中间。

这一章的三件事

- 一张对照表：哪些能力在跌，哪些在涨，哪些没变过
- 职业结构会变成哑铃：中间那截最难受
- 短期会先恶化，再来分化——别把"暂时的坏消息"当成"最终结局"

先说个招聘侧的观察。

有家公司这两年招人，HR 筛简历时碰到一件很怪的事：五年经验和两年经验的简历，内容高度重叠。

都是"熟悉某框架、负责某模块、使用某技术栈、完成需求若干"。区别只在年限和公司名。

面试官跟我讲过一个细节：他问一个五年经验的候选人，"你做过的东西里，哪个是你决定要做的？"对方想了半天，说"都是产品提的需求，我负责实现"。

他又问："那有没有哪次，你觉得这个需求不该这么做？"

对方说："有，但提了也没用，最后还是按产品说的做了。"

这不算回答错误。但这五年，在他的简历里体现不出来。

年限在涨，但可区分度没涨。这就是中间层最先出问题的原因：不是他们变差了，是他们的成长曲线和别人的重合了。

企业那边的算法其实很简单。招一个人，本质上是在买"产出除以价格"。当工具把年轻人的产出往上抬了一截，这个分母就变了——同样一份执行型工作，用三年经验的人加工具，和用八年经验的人加工具，产出差别不大，价格差一倍。

没有人需要做这个选择的时候，大家都相安无事。一旦需要收缩，账就摆在那儿了。

这跟公不公平没关系，这是算术。

二 一张对照表

把这一轮变化摊开看：

模 型

一张对照表

能力/产出	走势	说明
写代码的速度	跌	这是最先被接管的一层，没有悬念
熟悉语法、框架、API	跌	记得住不再值钱，知道"该不该用"才值钱
按文档实现需求	跌	目标明确、可验证，正是工具最擅长的
常规调试、改 bug	跌（较慢）	越标准的越先掉，越疑难的越晚
系统设计与权衡	涨	需要知道代价、风险、业务的优先级
定义问题、判断做不做	涨得最猛	输入不确定、结果不可立即验证

模 型

一张对照表（续）

能力/产出	走势	说明
懂业务、懂客户的真实处境	涨	隐性知识，不在文档里
跨部门推动、让人信你	涨	机器拿不到信任
对结果负责、敢拍板	涨	工具不承担后果
审查工具产出、兜住它的错	涨，而且是新增的活	产出变多，质检责任留在人身上
学习新东西的速度	没变，但更重要了	不是学工具，是学"新领域的判断依据"

这张表里有一条容易被忽略："写得快"跌得最狠，而它正是过去最容易被量化和考核的。

所以很多团队会出现一个怪现象：绩效考核里最看重的那几项，正好是贬值最快的那几项。

我见过一家公司的季度评优：拿 A 的是那个季度交付任务最多、bug 最少、从没延期的人，评得很服众。半年后那批活里，有相当一部分交给工具做了，一周就能跑完。没人觉得评错了，只是那个"最多"突然不再说明什么。

问题不在评得准不准，在于能数的都在跌，涨的那些不好数。代码行数、任务数、按时交付率，这三个最好统计，也贬值最快。而“砍掉了一个不该做的需求”“提前判断出这个方案会出事”这类贡献，几乎不可能进考核表——因为它是“没发生的事”，没法量化。

这就是眼下很多组织考核失灵的根源：不是不想改，是激励系统天然偏向可计量的东西，而可计量的那部分正在变得不值钱。

三 哑铃化

把上面那张表映射到人身上，形状是这样的。

一头是“能定义”的人。他们决定做什么、砍什么、做成什么样算好。人数少，价值高，AI 让他们更强。

中间是“能把定好的事做好”的人。过去这是产业的主力，数量最大。现在他们做的事正在被接管，而且被接管的顺序是从下往上，先是写代码，然后是拆任务、盯进度（这一层第10章会细讲）。

另一头是“能端到端交付”的人。一个人加工具，从想法走到上线，全流程自己扛。以前这需要一个团队，现在一个人能做到，前提是这个人能做全套判断。

中间塌陷，两头崛起。

两头的人到底长什么样？

模 型

哑铃化

	上面一头：能定义的人	下面一头：能端到端交付的人
日常在做什么	决定做什么、砍什么、什么算做好	从想法到上线全流程自己扛
手里的工具	用 AI 快速验证多个方案，挑一个	用 AI 补齐自己不会的技能
别人为什么需要他	少了没人知道该往哪走	少了这件东西就没人能交付
数量	少	正在变多
门槛	懂业务 + 敢担责	能做全套判断 + 肯对结果负责

中间那一截呢？不是消失了，是变薄了。过去一个十人团队里大概有六七个“把定好的事做好”的人，以后可能是两三个。

那多出来的人去哪了？三条路，都不轻松：

一部分往上走，学会定事：这需要时间和机会，不是想转就能转。一部分往端到端那一头走，自己能扛完整件事——这条对能力要求反而更高，因为要做全套判断。剩下的人留在中间，继续做执行型工作，但价格和可替代性都会慢慢变差。

留在中间不是不能活，是会很憋屈。你做的事没变，但你在组织眼里的稀缺性一直在掉。

但顺序要注意：短期先恶化。

为什么？因为工具最先让“初级产出接近中级”。一个刚毕业的人配上 AI，能做出过去两三年经验的人做出来的东西。企业算得很清楚：如果产出差不多，为什么要贵的那个？

于是短期内最难受的，不是刚入行的人，是“中等价位、中等经验、做中等难度执行活”的那批人——性价比最差。

老郑就是这个位置。

37 岁，做了十来年开发，去年底公司调整，他出来了。他原本觉得不难：技术扎实，项目经验多，怎么着也能找到。

半年里他投了几十份，面试十来家。他发现面试官总在问两类问题：

"你带过团队吗？""你能独立负责一块业务吗？"

他的回答一直是自己做过哪些系统、用了什么技术栈、扛过多大的量。

面试官点点头，然后就没下文了。

后来他去了一家小公司，职级降了一级，薪水也降了。

有意思的是后面这一年。小公司人少，没人给他拆需求，他被迫从“接活”变成“自己想该做什么”。头几个月很不适应，他跟我说的原话是“每天最痛苦的不是写代码，是要决定今天写什么”。

一年之后他跟我说了一句挺不一样的话：

"以前我以为我卖的是技术，现在我知道我卖的是'这块东西交给我，你不用管了'。"

他自己说，他攒的东西没变，是卖法变了。

他那半年最难的不是钱。是有一次他反复在想：我这十几年攒下来的东西，好像没人想要。

他攒下来的确实是好东西，问题是那些东西属于"能做好别人定好的事"这一类。而这一类是当下最不缺的。

但短期的坏消息不等于终局。分化的前提是：有人把经验转成了判断力，有人没有。这一条第3章展开。

四 别让自己长成工具人

中间那截人是怎么长出来的，这一节单独说清楚。

这个词不好听，但它描述的是一种很常见的处境，值得单独说一说。

先定义清楚。工具人不是"被别人当工具使"的意思，那个是抱怨。这里说的是一种长法：长期只做执行、不做判断，慢慢长成那个形状。

第1章的小周就是。他不懒、不笨，谁卡住了找他都能解。问题是他工作输入永远是清楚的——需求、接口、验收标准，别人都给好了。他做得很漂亮，但里面没有一步是他定的。

这么干三年五年，会出两件事。

第一件，判断力没有练习样本。判断力不是懂道理就有的，是做过决定、担过后果才长出来的（第6章细讲）。一个人如果连续五年接的都是"已经定好的活"，那五年里他一次都没练过，年限涨了，能力曲线是平的。

第二件，他的产出越来越可描述。这一点最要命。

可替换性跟能力水平关系不大，跟"工作能不能被清楚描述"关系很大。一份工作如果输入输出都写得明明白白，那任何人、任何工具，只要能接住这套输入输出，就能替换掉你。你说不清自己哪里不一样，是因为确实没有东西不一样。

所以工具人的处境不是"被淘汰"，是"没有理由被留下"。这两句话听起来像一回事，其实不一样：前者是别人做了什么，后者是你拿不出东西。

由此带来一个更难熬的地方：上限越来越低。

执行型路线的加薪靠什么？靠年限、靠跳槽、靠行情。这几样都不由你控制，而且都有天花板。判断型路线不一样，它的差距是拉开的，越往后越明显，因为它攒的是复利。

我见过最拧巴的情况是这样的：一个人明明更努力了、产出更高了，却越来越不敢谈条件。因为他心里清楚，自己多出来的那部分产出，换个人来也一样。

怎么知道自己有没有在往这个形状长？有个很土但很准的办法。

把上周做的事列出来，逐条问一句：这件事换一个技能相当的人来做，需要知道什么额外的背景吗？

如果需要，说明你身上有别人没有的上下文。如果十条里有八九条都不需要，那你卖的就是通用产能，而通用产能是最先被比价的那一类。

破解的办法不是辞职，也不是立刻转岗。是在现在这份活里，把判断的部分一点一点抢回来。从第6章开始讲的就是这件事。

五 走的人，和留下的人

老郑走的时候，组里没人觉得是坏事。三个月后，剩下的人开始感觉到区别。

他们组从七个人变成五个，需求没减。leader在会上说"工具用起来之后效率上来了，我们顶得住"。大家点点头，没人反驳，因为这话在数据上确实成立——交付量没掉。

但有几件事悄悄变了。

隐性知识走了。老郑在的时候，某个老接口的坑他记得，某次线上事故的处理路径他脑子里有。这些东西没写进文档，走了就是走了。后面再碰上，得重新摸一遍。

每个人的上下文变宽了。七个人分的事现在五个人分，每个人手上的系统多一块，切换成本上来了。事情没有变简单，只是变杂了。

没人敢说“做不完”。因为“效率上来了”这句话已经定下调子。你说做不完，等于说你没用好工具。

于是出现一个很怪的现象：数据上这个组更高效了，里面的人却普遍更累、更焦虑，而且不敢说。

这不是个例。这就是“提效不减负”在团队层面的样子。

所以看这轮变化，别只盯着“有没有人被裁”。还要看另一件事：留下的人是什么状态。一个团队可以同时做到“产出上升”和“所有人都在硬撑”，而且这两件事会互相强化——产出上升证明了还能再压。

六 这个过渡会有多长

有个很实际的问题：这是明天的事，还是十年后的事？

我的判断是，它已经开始了，但会很不均匀，全部走完大概要五到十年。这个数字没有可靠依据，是我从接触到的情况推的，当个量级看就行。不均匀体现在三个地方：

行业上。数字化程度高、产物本来就在电脑里的，走得快——软件、内容、数据分析、翻译。需要物理接触或者强监管的，走得慢——医疗、教育、制造现场。

公司上。敢用工具、流程和考核跟着改的公司，一两年内就能看到结构变化。保守的公司可能三五年后才感到压力。而个人感受到的冲击，取决于你所在的那家公司走了多远，不取决于行业平均。

岗位上。同一个公司里，做标准业务的组先变，做疑难系统的组后变；写新代码的先变，维护老系统的后变。

所以别用"行业还没动静"来安慰自己。你该看的是你自己那个组、那个岗位，过去一年里"定好的活"是不是在变少。这个信号比任何报告都准。

另外提醒一句：不均匀也意味着有人会先跑出来。差距不是同时拉开的，是分批的。早两年动的人，吃到的红利和晚两年动的人完全不是一回事。

七 别急着对号入座

有个提醒。

看这类对照表，人很容易立刻给自己判个位置，然后要么恐慌，要么松口气。两种反应都不对。

第一，涨跌不是非黑即白。"写代码速度"在跌，不等于"不写代码"。反过来，现在写得更多的人价值更高，因为实现便宜了，可以大胆试、快速验证。跌的是"写得快"这件事作为卖点的价值，不是写作本身。

第二，你的位置不由岗位名称决定，由你实际做的判断决定。一个 title 是高级工程师、但每天只做定好的活的人，实际在中间层；一个 title 是普通开发、但经常在定方案拍板的人，实际在上一头。

第三，这一轮是慢性的，不是事件。没人会通知你"你的技能过期了"。更常见的过程是：你的产出明显上来了，但你的处境一点没变好——活更多、人更少、说不清自己好在哪；或者像老郑那样，直到出来找工作才发现自己的经验没人要。等你意识到的时候，通常已经过了一两年。

第四，别用"我还很忙"来判断自己安全。忙是团队的状态，不是你的价值证明。眼下很多团队是"人少了、活多了、每个人都很忙"，而这跟"每个人都很可替代"完全不冲突——上一节说的那种处境，恰恰在全员都忙的时候最难被发现。

自检三问

- 1 你的简历里，有没有一件事是"你决定要做的"，而不只是"你参与实现的"？
- 2 过去一年，你的产出是不是明显提升了？如果是，你的处境（话语权、不可替代性、时间安排）有没有跟着变好？
- 3 如果你明天要找工作，你能不能用一句话说清"我能负责什么"，而不只是"我会什么"？
- 4 把上周做的事列出来，有多少件是"换一个技能相当的人来也能这么做"的？

可以做的事

个人

- 1 把你的能力按上面那张表分类，看看你最花时间的三项落在哪一边。如果全在左边，这是个信号
- 2 挑一件事往"定义"那一侧挪半步：不是等需求，而是自己提一个需求，写清楚为什么值得做
- 3 给自己准备一个"能负责什么"的版本：一页纸，讲清楚你能独立拿下哪一块，而不是罗列技术栈

管理者

- 1 检查一遍团队的考核项。凡是只用"数量、速度、按时率"的，都在激励正在贬值的東西
- 2 招聘时别再拿"年限"当能力代理。问一句"哪件事是你定的"，区分度比技术题高
- 3 团队里那些"中等经验、执行型"的人，是最该被提前拉一把的。别等他到了老郑的位置才谈
- 4 效率增益别全收走。留出一部分还给团队——哪怕只是"少接一点需求"。短期看是少赚了，长期看，一个全员疲惫且不敢说话的团队，迟早会在某个没人预料到的地方出事

别的岗位呢

中间层塌陷不是软件行业独有。

会计里做账的那一段最先被自动化，而"怎么设计核算口径、怎么处理税筹"的那一段在涨。翻译行业更明显：执行层的笔译被吃掉大半，而"判断这个译法在目标市场会不会惹麻烦"的人更值钱。法务里起草合同的活在被替代，"这单生意的条款该怎么谈"没有。

规律是同一条：凡是能被写清楚规则、并且结果能立刻验证对错的部分，都在往下走；需要判断、需要担责、需要理解人的部分，都在往上走。

35 岁线的真相

35 岁不是年龄的坎，是"经验没转成判断力"的坎。

这一章的三件事

- 市场一直在为"执行"定价，而执行的价格随年龄往下走
- 真正的分岔在十年前就发生了：有没有人让你做过决定
- 但也要诚实：结构性的压力是真的存在，不是一句"提升自己"就能抹平

阿凯和老林是同一年进的同一家公司，同一个组，同一个学校的师兄弟。

头两年他们几乎没区别：都是来得早走得晚、代码写得干净、出了问题能顶上。组里谁有急活，喊一嗓子两人都上。

分岔发生在第五年。

那时候组里要换一套消息队列，leader 在会上问了一句："这个选型谁跟一下？"

阿凯没接。他后来跟我说，他当时想的是：这种事最后都是 leader 拍，我跟着干就行，万一选错了还得背锅。

老林接了。他花了两周做对比测试，写了一份选型报告，最后拍了板，说"出问题我负责"。

那套东西上线四个月出过一次事故，凌晨三点，流量洪峰把队列打爆了。老林爬起来查到天亮，第二天写了事故复盘，在会上说"是我选的，我判断错了容量"。

那半年老林过得不太好，口碑掉了一截。leader 没换掉他，只说了一句"接着干"。

第八年，老林开始带一条业务线。第十年，他管着三四十号人，开始跟客户谈需求、决定这条线做什么不做什么。

阿凯第十年还在写代码，也是组里最靠谱的那个。谁有急活还是找他，他也还是能顶上。他的职级在第七年之后就没动过。

去年他们公司有一轮架构调整，阿凯的那个组被合并了。他被调到一个新组，title 没变，活变少了。

他有一次喝酒说了句挺实在的话：

"我这十年做的，都是别人已经决定要做的事。"

我问过他接下来打算怎么办。他说想把技术再捡一捡，学点新的东西——云原生、AI 工具那一套。

我说那些东西二十来岁的人学得比你还快。他没接话。

这句话我记了很久。十年，不短。但如果这十年里你一次都没做过选择，那这十年在市场上是什么？是一个更贵、更累、牵绊更多的执行者。

二 市场到底在为什么付钱

35 岁线这件事，大家习惯从"歧视"的角度去讲。歧视当然存在，但这不是主因。

主因很枯燥：市场在为"执行"定价，而执行的价格随年龄下降。

拆开看，一个执行者的价值由三样东西决定，能做多久、做得多快、要多少钱。

模 型

市场到底在为什么付钱

	25 岁	35 岁	45 岁
能投入的时间精力	高	中，且在往下降	低
熟练度带来的速度	中，涨得快	高，接近天花板	高，但不再涨
市场价	低	高	最高

三条曲线在三十岁以后开始交叉：速度到顶不再涨，投入能力开始掉，而价格还在往上走。

所以真正的不是"35 岁不要你"，是"35 岁的执行太贵了"。

这里要补一句：过去这条曲线一直存在，为什么没那么疼？

因为过去执行能力也在涨。三十五岁的工程师虽然贵，但他做得更快、更稳、能扛更复杂的活，涨的那部分能覆盖价格的增長，买卖双方都划算。

工具打破了这个平衡：它把"做得快、做得稳"这一项在年轻人身上也拉起来了。于是三十五岁的工程师剩下的溢价，只剩"能扛更复杂的活"。而这一项，需要的是判断力，不是熟练度。

而这件事在 AI 出现之前就已经成立。AI 做的是把这条曲线往下压——它让"熟练度"这一项在年轻人身上也能快速拉起来，于是交叉点提前，而且压得更狠。

那为什么有人过了 35 岁反而更好？

因为他们卖的早就不只是执行了。

他们卖的是判断，这个需求该不该做、风险在哪、怎么权衡。而判断力这东西，偏偏是随年龄往上走的，因为它靠的是见过的"决策—结果"对。

还有一件事得说清楚：当组织决定减员，先减的是谁？

不是最差的，是性价比最差的——贵、可替代、做的事别人也能顶。三十五岁以上、做执行活、薪资在中高位的人，正好落在这个位置。

所以"提效不减负"和"35 岁线"其实是同一件事的两面：留下的人更累，而走的人往往先从中年开始。

所以同一个 35 岁，对不同的人是两件事：

- 执行型：价格往下走，价值也往下走 → 坎
- 判断型：价格还在往上走，但价值涨得更快 → 分水岭

一句话说完：35 岁线不是年龄线，是"你的经验有没有转成判断力"的那条线。转了的，年龄是资产；没转的，年龄是负债。

三 为什么判断力偏偏随年龄涨

这里有个反过来的好消息，值得单独说。

执行能力随年龄往下走，判断力却相反：它靠的是你脑子里存了多少个"决策—结果"的样本。

见过三次上线事故的人，下次评估风险时心里有底；谈崩过两个客户的人，知道哪种承诺不能随便给；砍过需求也砍错过需求的人，分得清"看起来该做"和"真的该做"。

这些东西没办法速成。它只能靠一次又一次真做、真错、真复盘攒出来。二十岁的人哪怕再聪明，样本量也不够。

所以年龄对判断力是资产，对执行力是负债。这就是同一条 35 岁线，在不同人身上是两件事的原因。

但要说清楚一句：年龄本身不产生判断力，产生判断力的是"做过决定并看过后果"。如果只是把同一年的经验重复了十年，那只是样本量为 1 的十年。阿凯就是这种情况。

四 为什么大部分人转不成

这里有个挺残酷的地方，得说清楚。

判断力不是看书看来的，也不是干得久就自动有的。它只能靠"做决策、然后承担后果"来练。你得真的拍板，然后眼看着结果好坏落在自己身上，下次才知道边界在哪。

而这种机会，过去组织只发给架构师和总监。

一个普通的开发工程师，干到第十年，可能一次都没真正拍过板。不是因为他不行，是因为没有机会。他的每一天都是"需求已经定了，你来实现"。日子久了，他自己也习惯了——习惯了等别人拿主意，习惯了"我负责做好"这句话里的安全感。

阿凯不是不想，是没人让他想过。

所以这事的根子在两头：个人没有主动要，组织也没有给。

主动要，是有风险的。你接了选型，出事就是你的。老林那半年过得不好是真实的代价。很多人算过这笔账，觉得不划算——做好执行，稳稳当当，不好吗？

短期看确实划算。长期看，这是把十年的经验存进了一个持续贬值的账户。

五 也得诚实：结构性的压力是真的

写到这儿得停一下，说几句不那么提气的话。

上面这套说法，很容易被读成"被淘汰是因为你不努力"。这个我不认，得讲清楚几个真实存在的结构性因素：

第一，岗位总数确实在收缩。执行效率提升之后，一部分公司做同样的业务需要的人变少了，而且这是结构性的，不是周期性波动。你等不来"行情好转"。

第二，年龄偏见真实存在。有些公司就是不招 35 岁以上的一线，简历筛选那一关就过不去。你再有能力，连面试机会都没有。

第三，成本结构是硬约束。一个家庭、房贷、孩子，决定了你不能随便降薪去换赛道。年轻时可以"先去试试"，四十岁试错成本高得多。

第四，转型需要时间，而市场不等你。判断力要三五年才练得出来，但账上的压力是每月都来的。

第五，最麻烦的一条：你可能连转型的力气都没有。前面讲要练判断力、要复盘、要学业务，这些都要精力。而眼下的现实是很多人已经很累了，下班只想躺着。这不是意志力问题，是真实的资源约束。

这一段我写得很不愿。写进去，整本书就不那么提气；删掉，它又变成了那种"只要你努力就行"的腔调。

所以后面几章给的动作，我都尽量压到最小——不是让你在现有负担上再加码，是先在现有工作里换一种做法。但也说实话：一点不加是不可能的，至少开头得挤出一点。关键不在加多少，在加在哪个维度上。

所以这本书不是在说"只要你努力就不会被淘汰"。我想说的是另一件事：

结构性因素你改变不了，能改变的是你在哪个池子里。同样一轮收缩，先被砍的一定是"贵而可替代"的那一批。把自己挪出那个池子，不是保证安全，是把概率往自己这边拨一点。

以及在最坏的情况下——如果你真的被优化了一个判断型的人，重新站起来也比执行型的人快。因为市场上缺的是能定事的人，不缺能干活的人。

六 那现在还能做什么

不管你今年二十八还是三十八，可以做的事是同一件：开始积累"决策—结果"的经验样本。

具体一点：

- 主动接一次要拍板的活。不用大，一个技术选型、一个方案取舍就够。关键是你说了算，并且明确说"后果我担"
- 事后一定复盘。判断力长在复盘里，不长在做决定那一刻。做对了要知道为什么对，做错了要能说清错在哪一层
- 把经验挂到业务上。光有技术判断不够，得知道这个决定对客户、对收入意味着什么。这是下一部分的重点
- 别等组织给你机会。组织给的机会永远不够分。自己造，哪怕只是在一个小项目里做一次完整的选择

组织那一边能做的，是把决策权往下放，并且允许犯错。这一条在第12章细说。

不同年纪起步，重心不太一样：

三十以前，重点是攒样本。别怕接小决定，一个技术选型、一次排期取舍都算，关键是你要说清为什么这么选，事后有复盘。

三十五左右，重点是把经验挂到业务上。这时候你已经有技术判断了，缺的是"这个决定对客户、对收入意味着什么"。这一块是下一部分的重点。

四十以上，重点不是补短板，是找到那个"只有你能定"的位置。这个年纪再去比手速没有意义，你的价值在于别人拿不准的时候，你能拿得准。

自检三问

- 1 过去一年，你做过几个"后果算你的"决定？有没有一个？
- 2 如果把你的年龄和薪资都抹掉，只看你做的事，你和刚工作三五年的人有什么不一样？
- 3 你所在的位置，是"离开了会缺一种能力"，还是"离开了会缺一个人手"？

可以做的事

个人

- 1 这个季度主动接一件要拍板的事，写清楚"我判断什么、错了怎么办"，然后真的担下来
- 2 做一次十年盘点：把你做过的决定列一列，看看是"实现的决定"多，还是"该不该做"的决定多
- 3 找一个正在做判断的人，看他在会上是怎么想、怎么说的。这东西靠观察学比靠读书快

管理者

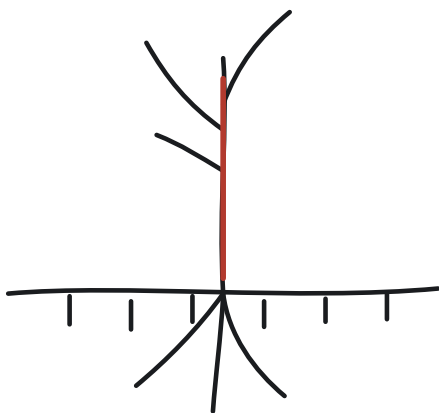
- 1 检查一下你手里握着多少决定权。能下放又不会出大事的，尽快下放：这是团队唯一能练判断力的途径
- 2 有人拍板错了，别急着换人。只要复盘是诚实的，那次错误的成本远低于"从此没人敢拍板"
- 3 35岁上下的执行型老人，是团队里最该被提前谈话的人。别等到调整那天
- 4 真要减员，先想清楚按什么顺序。只按价格和年龄砍，砍掉的不只是成本，还有稳定性——留下的每个人都会开始算"下一个是不是我"，而这个念头比加班更消耗人

别的岗位呢

35 岁线最早是 IT 行业的说法，但它不是 IT 独有的。

会计、翻译、法务、设计、媒体，凡是"靠熟练度吃饭"的岗位，都在同一条曲线上，只是晚几年。有意思的是，医生、律师、教师这些职业反而不太焦虑——不是因为他们更聪明，是因为他们的培养体系里很早就强制要求做判断并承担责任。住院医要独立值夜班，年轻律师要独立见客户。他们从二十几岁就在练判断力。

IT 行业的问题在于，我们设计了一套让人可以十年不做决定也能拿到不错回报的体系。这是行业的锅，不完全是个人的。



PART TWO

第 二 部 分

立身

价值的三个来源

价值公式

为什么同样是 AI，产出差三倍

工具是公开的，差距只能来自使用它的人。

这一章的三件事

- 一个公式，把"我有没有未来"这个问题翻译成能算的东西
- 括号里如果是零，乘数再大也是零，这解释了纯执行者为什么会归零
- 反过来说也是好消息：同样的判断力，现在能撬动十倍的产出

去年年底，一家做企业协作 SaaS 的公司做季度复盘，出了个挺有意思的数据。

同一个产品组，八个人，用的是同一套工具，账号还是同一个月开的。季度末统计交付：最多的人做了 15 个需求，最少的做了 5 个，正好三倍。

这里得先把"一个需求"说清楚，不然这个数字没法看。这家公司的需求指的是产品评审过会、进了迭代排期的条目，规模都不大，多数在两三个人天到两周之间。同样八个人，如果做的是交易、结算这类核心系统，一个季度人均两三个是正常的；如果做的是 C 端那种小步快跑的迭代，一个季度四五十个也不稀奇。所以别把 15 这个数往自己团队上套，值得看的是差距本身。

按常理，做 15 个的那位该拿 A。结果季度评优给了做 5 个的。

组里有人不服，去找 leader 问。leader 把两个人的东西摊开看了一遍。

做 15 个的叫阿哲，工作三年。他的流程很标准：需求来了，写清楚，交给工具，出来以后改一改，跑通，提交。一个接一个，节奏很稳，从没延期。

问题在后面。那 15 个里，有 6 个上线之后基本没人用：后台数据看得见，日活是个位数。有 2 个上线三个月后被重构成了别的做法，因为最初的方案没考虑跟另一个系统的对接。剩下 7 个是常规功能，做好了，没什么可说的，也没什么可错的。

做 5 个的叫老唐，工作九年。他这个季度一共接到 10 个需求。

他在评审阶段砍掉了 5 个。其中一个给老客户加导出功能。他算了算：那批客户一年导出不超过两次，手工导一次五分钟，做这个功能要三天。他写了个说明，问了一句"这三天是不是有更该花的地方"。产品那边同意了。

剩下 5 个他做完了。其中 2 个后来成了第二年主打功能的底座，另外 3 个至今在用。

还有一件事不在统计表里：他花了大概两周，把组里那套老接口的调用方式重做了一遍。那两周他一个需求都没交付。重做之后，后面所有人的活都快了一截，包括阿哲。

所以那"三倍"到底怎么算？

按交付个数，阿哲是老唐的三倍。按"现在还在用、且真有人用"算，反过来。按"这个季度之后这个组变好了多少"算，差距还要更大一点。

阿哲后来跟我说过一句挺实在的话：

"我那个季度挺拼的，每天都有东西交出去。我真不知道差在哪。"

他差在把工具当成了一台更快的打字机。老唐把它当成了一个"不花时间就能验证想法"的东西。

二 一个公式

这件事可以用一行式子说清楚：

$$\text{个人价值} = (\text{判断力} + \text{领域深度}) \times \text{表达带宽} \times \text{AI 能力}$$

四块，一块一块说。

AI 能力，是乘数。它是历史最大值，而且还在变大。但有一件事得看明白：它是公开可得的。你的同事、你的竞争对手、刚毕业的年轻人，用的都是同一批工具。所以它不是你的护城河，它是所有人的水位。

水位上涨的时候，个子矮的先淹。

判断力，是知道该做什么、不该做什么、做成什么样算好。第 6 章整章讲它，这里先放着。

领域深度，是你对这块业务的 know-how——这个系统为什么长成这样、客户真正烦的是是什么、哪些坑前人踩过。第 5 章讲它。

表达带宽，是你能把脑子里的东西说清楚到什么程度，对人和对 AI 都算。第 8 章整章讲它。

这四块里有三件事值得单独说。

第一，括号里趋近于零的时候，乘数再大也是零。

这就是纯执行者的处境。他的括号里没什么东西，不需要判断该不该做，也不需要懂业务，工具给什么就交什么。这时候 AI 能力再强，乘出来的数还是接近零。

所以“我会用 AI”这句话本身不值钱。值钱的是你拿它去放大什么。

第二，反过来也是好消息。

同样的判断力，现在能撬动以前十倍的产出。老唐过去要花两周验证一个想法，现在两小时；过去砍一个需求得先说服所有人“这东西做出来也就这样”，现在他可以先做个原型给人看。

所以这一轮不是“所有人一起变差”，是差距被放大。这也是为什么我在序里说，接下来十年是这行里个体与个体差距拉得最大的十年。

第三，工具越普及，括号里的东西越值钱。

这是个推论，但很关键。当所有人都会用工具，工具带来的那点领先就互相抵消了。剩下的差异，完全由括号决定。

你可以自己验证：两年前“会用某个 AI 工具”还是个能拿出来说的技能，现在呢？它变成了默认项。默认项不加分，但不会扣分——直到某天连这个也不再需要提。

还要说一句为什么要写成加号。判断力和领域深度不是互相依赖的关系，是互相喂养：深度不够的时候，判断力可以靠通用常识补一部分；判断力不足的时候，深度至少能让你提供可靠的依据。它们能互补，但不能互替。所以用加号，不用乘号——用乘号的话，任何一项为零整个人就归零了，那不是事实。

三 阿哲差在哪

把阿哲那个季度按公式拆开看：

模 型

阿哲差在哪

	阿哲	老唐
AI 能力	一样（同工具，同样熟练）	一样
表达带宽	不低，写得清楚，工具产出质量不差	不低
领域深度	薄，不知道那 6 个需求谁会用来干嘛	厚，跟业务方熟
判断力	基本没用上，从没问过“该不该做”	砍了 5 个，还重做了接口

他的问题不是不会用工具，是括号里几乎是空的。15 个需求的产出，等于把一个接近零的基数放大了十倍：还是接近零。

老唐交付少，是因为他把乘数用在了别的地方：砍需求之前先做原型验证一下，重做接口之前先算收益。

所以这里有个比公式更细的东西：

乘数用在哪个环节，比乘数有多大更重要。

用在执行上，得到的是更多的执行。用在判断上，得到的是更准的判断。

这一条正好接上第 1 章那件事——你把 AI 省下来的时间全部投入执行，结果只是把跑步机调快了一档。跑得越快，标准抬得越快。

阿哲那个季度就是这样：他确实更快了，于是接到了更多需求，于是更快。季度末他很累，也很委屈。

四 三个常见的误判

误判一："我工具用得比他熟。"

工具正在变得越来越傻瓜化。两年前还要琢磨怎么问，现在说句人话就行。熟练度红利会消失，而且消失得比你想象的快。

误判二："那我多学几个工具。"

乘数已经很大了。继续往上加乘数，边际收益远不如补括号里那两项。学过三个工具的人和学过五个工具的人，差距很小；懂业务的人和不懂业务的人，差距是数量级的。

误判三："等我判断力上来了再用。"

反了。工具本身就是练判断力最便宜的场地。

这一条值得多说。过去判断力难练，不是因为人笨，是因为试错贵。你要验证"这个需求做出来是不是也就这样"，得真的做出来，两周。没人肯花这两周，所以大家只能拍脑袋。

现在这个验证两小时就能做。老唐能在评审阶段砍掉那 5 个需求，靠的就是先花两小时弄个能跑的原型，摆到产品面前说"你看，就是这样"。

工具不只是放大器，它还是训练器材。它把练判断力的学费从两周降到了两小时。

五 乘数用在判断上，长什么样

道理讲完了，说点能直接用的。工具不只是能帮你更快地写，它还能帮你更好地想。四个用法，我今天就在用。

第一，让它扮演反方。

做完一个方案，问一句："如果这个方案是错的，最可能是哪一步？"

你会发现它给出的角度，往往是你自己想不到的那一类——不是技术上的，是场景上的：某个极端情况、某个你没考虑的用户群、某个时间点。

这一步过去得找人评审，还得找个敢说话的。现在随时能做，而且它不会顾及你的面子。

第二，让它帮你算代价。

三个候选方案，让它分别列出：要动哪些东西、依赖谁、哪些地方会牵一发而动全身、最坏情况是什么。

这不是让它替你选，是让你在选之前把代价看全。很多判断错，不是选错了，是没看见代价。

第三，快速验证想法。

第 3 节讲过，这是最值钱的用法。两小时做个能跑的东西，摆给人看。验证的价值不在于它做得多好，在于它能把讨论从"想象"拉到"实际"。

老唐砍那 5 个需求，靠的就是这个。

第四，让它帮你把模糊的想法写清楚。

你脑子里有个判断，但说不出来。跟它说个大概，让它整理成三条，然后你再改。改完之后那三条就是你的判断——写下来可以给人看，也可以存档（第 6 章那条）。

这四个用法的共同点是：它不替你判断，它让你判断得更快、更全、更有依据。

一句话：把工具用在"想"上，比用在"做"上值钱得多。

六 那括号怎么加

三个方向，后面三章各讲一个：加判断力（第 6 章）、加深度（第 5 章）、加带宽（第 8 章）。

至于乘数，别再花力气了。已经够大了，而且你加不赢，那是所有人的公共水位。

一句话收口：别问"我还能用什么工具"，问"我用工具放大的是什么"。

七 再往前推一步

有个问题现在就得想：这个乘数还在变大。再大十倍会怎么样？

我的判断是，被接走的范围会继续往里扩，边界卡在"能不能验证对错"上。

现在还留在人手里的很多活——写测试、做迁移方案、排查线上问题——之所以还得人做，不是机器不会，是结果对不对暂时还得人来判。哪天验收标准能被写清楚、能被机器自己检查，这部分也交出去了。

所以括号里那两项会被迫再往上挪一层。判断力从"这几个方案选哪个"，挪到"这件事到底要不要做"；领域深度从"我懂这个系统怎么搭的"，挪到"我懂这块业务为什么存在、靠什么赚钱"。

说白了，那条安全线在往后退。你不必跑得比它快，但方向得对——往前挪，不是往快里练。

推到头，稳的位置只剩两个：离客户最近的那个，和离后果最近的那个。前者知道要做什么，后者出了事跑不掉。那些既不碰客户、也不担后果的位置，不管眼下多忙，都在被稀释的路上。

听着耳熟就对了。第 2 章讲哑铃化，第 10 章讲中间层被掏空，这里是第三种说法。同一件事换个角度讲三遍，是因为它确实是这轮变化里最硬的那块。

自检三问

- 1 上个季度你交付的东西里，有多少现在还在用、且有人用？
- 2 工具给你省下来的时间，你投回到哪了？是接了更多活，还是多想了一步？
- 3 如果明天全公司都换上更强的工具，你比同事强在哪？

可以做的事

个人

- 1 做一次"有效产出"盘点：把过去半年做的东西列出来，标上"还在用 / 没人用 / 被重做了"。比例会告诉你很多
- 2 下次接到需求，先花两小时做个最小验证，拿它去问"我们要的是不是这个"。这一步就是判断力的训练
- 3 这季度别再学新工具了。把时间拿去搞懂你做的这块东西给谁用、解决什么

管理者

- 1 别再按交付个数评优。加一条"上线之后的使用情况"，数据自己会说话
- 2 允许有人把时间花在"不交付"的事上：比如老唐那两周的接口重做。这种投入在短期报表里是负的
- 3 团队里工具用得最熟的那个人，未必是最该被奖励的那个。看看他放大了什么

别的岗位呢

这个公式跟写代码没关系。

设计师的价值 = (审美判断 + 对用户和场景的理解) × 表达带宽 × AI 能力。产品经理的价值 = (需求判断 + 对用户和市场的理解) × 表达带宽 × AI 能力。分析师的价值 = (指标判断 + 对业务的理解) × 表达带宽 × AI 能力。

凡是"靠专业知识产出"的岗位，都是这个式子。区别只在括号里填什么。

乘数人人有，括号各自不同。这就是接下来十年人和人拉开差距的地方。

业务理解

为什么它成了立身之本

技术决定你能不能做出来，业务决定你做出来的东西有没有人要。

这一章的三件事

- 判断的依据是业务理解，而 AI 留下来的正是判断——所以业务理解成了立身之本
- 五层阶梯，可以自测你现在站在哪一层
- 一个必须说清的边界：技术没有贬值，是转移了。最稀缺的是"业务 × 技术"

— 小许做了四面镜子

小许做 CRM 四年，技术上没得说。标签体系、实时计算、画像自动更新，做得漂亮。

去年他们做了个"客户画像"功能。需求是产品提的：给每个客户打标签，行业、规模、活跃度、意向度，实时更新。销售打开客户详情页就能看见。

小许做得很好。上线那天他还发了条朋友圈。

三个月后他去看数据：打开率不到两成。打开的人里，大部分停留不到十秒。

他不明白，去找销售聊。销售总监说了句让他愣在那儿的话：

"我要的不是知道这个客户是什么样的人。我要知道我下周该联系谁。"

他做的东西没错，只是不是人家要的。人家要的是一张"今天该打给谁"的清单，他给的是一面镜子。

镜子做得再清楚，也回答不了"下一个打给谁"。

后来他被安排跟着销售跑了两个星期。回来之后他改了做法：先做排序，谁最可能成交、谁快流失了、谁该续费了；标签退到后面，只作为排序的依据藏在底下。

第二版上线，打开率涨了好几倍，销售开始每天用。

他跟我说，那两周学到的东西比前四年多。

"我以前以为我懂这个系统。其实我只懂这个系统是怎么搭起来的。"

小许不是不努力，也不是技术差。他只是从来没见过用他做的东西的人。

二 一个被拦下来的需求

再说一件相反的事。

老谭做研发的年头更久，现在是那条业务线的技术负责人。有次评审，产品提了个需求：给大客户做自定义报表，客户可以自己拖字段。

老谭问了三个问题。

哪几家客户提的？——三家。

他们现在怎么解决？——销售手工导出数据，做成 Excel 发过去。

我们做完之后，谁去教他们用？：没人答得上来。

老谭说，那这个我不做。三家客户，每家一个月要一次，销售做一次二十分钟。做这个功能至少两个月，而且做完之后大概率还是销售在做，因为客户懒得学。不如把导出的数据格式调好，让他们做 Excel 更快。

产品当时不太高兴。三个月后那三家客户都续约了，没人再提报表的事。

老谭跟我讲这事的时候说了句挺有意思的话：

"砍需求比做需求难多了，因为你得知道砍了会怎样。"

注意这句话——"知道砍了会怎样"。这不是技术能力，也不是态度。它需要你知道这块业务怎么运转、客户真正在意什么、有没有更省事的替代方案。

这就是业务理解。

三 为什么偏偏是它

回到第 1 章那条线：AI 吃掉翻译，留下判断。

那判断从哪来？

判断不是凭感觉。你说"这个需求不该做"，得有依据。依据从哪来？从你对这块业务的理解来——谁在用、用来干嘛、不做会怎样、有没有更省事的办法。

所以完整的链条是这个：

AI 吃掉翻译 → 留下判断 → 判断靠业务理解 → 业务理解成了立身之本。

这条链的最后一环，是这本书最想说的一件事。我在这个行业里见过很多人走得很远，也见过很多人卡在半路，回头看，分岔点往往不在技术上，而在他们有没有往业务那一侧走过去。

年轻人最容易误解的一点：以为"把代码写好"就是全部。不是。把代码写好是入场券，不是终点。

四 为什么 AI 拿不走它

可能有人会说，AI 也能学业务知识。它能读完你们所有的文档、需求单、行业报告，比任何人都熟悉。

能。但有三样东西它拿不走。

第一，业务的"为什么"不在文档里。

文档里写的是结论，不是过程。"这个字段必填"——为什么必填？因为两年前有个客户填漏了，对不上账，最后赔了一笔钱。这类故事不写在任何文档里，它活在人的记忆里，活在组织的历史里。

AI 读得到结论，读不到那次赔钱。

第二，业务是变化的，而且充满利益冲突。

需求从来不是客观事实，是多方博弈的结果。销售要的是成单快，财务要的是回款稳，客户要的是便宜，老板要的是增长。同一个需求在不同人嘴里讲出来是不一样的。

判断"哪个才是真需求"，得知道各方的处境和筹码。这些东西不在任何系统里，AI 也拿不到。

第三，也是最实在的一条：AI 不会说"不"。

你让它做，它就做。它不会拒绝，也不会替你承担后果。

而真正懂业务的人会说"这个不该做"，因为他知道做起来也没人用，或者做完了要赔钱。说"不"是有代价的，你得顶住压力，还得给出站得住的理由。

一个从不说"不"的人，谈不上懂业务。

五 五层阶梯

"懂业务"这三个字太模糊了，会被说成一句正确的废话。所以拆成五层，你可以自己量一量。

模 型

五层阶梯

层级	是什么	典型的话
L1	知道功能是什么（会用）	"这个功能是这样的，点这里能导出"
L2	知道流程怎么走（懂流程）	"这个流程要走三步，先建单、再审核、最后归档"
L3	知道为什么这样设计（懂权衡）	"这里之所以要人工审核，是因为上次自动通过出过事"
L4	知道客户真正的痛点和替代方案（懂需求源头）	"客户真正烦的不是导不出数据，是每次都得导"
L5	能判断这盘业务该往哪走（懂方向）	"这块明年要从卖授权转订阅，所以现在做的东西得能按量计费"

大多数人在 L2。这没什么丢人的，但得知道自己在哪一层。

自检标准只有一条：你能不能说"这个需求不该做"，并且说服对方。

说不出来，或者说了但说不出理由，那基本还在 L2 以下。这条标准很硬，也很容易验——下次评审会你就可以试。

还有一点要说清楚：不是人人都要到 L5。

L5 是少数人的位置，它需要的不只是理解力，还需要信息量 and 责任。对大多数人来说，现实的要求是把自己所在那一环的业务理解到位，也就是 L3 到 L4。

写代码的要知道这个接口的调用方是谁、为什么必须这么设计；做测试的要知道这个场景下用户最容易在哪一步出错；做运维的要知道这个系统挂十分钟对业务意味着什么。

每个人都把自己那一环往 L4 推一推，整个组织的判断力就上来了。

六 听业务的话，不等于懂业务

这里有个坑，很多人掉进去还不知道。

很多人以为“懂业务”就是照着业务方说的做——销售说要什么就做什么，客户提什么就实现什么。这不叫懂业务，这叫传声筒。

业务方说的，往往是解法，不是问题。

有个例子我一直记得。一家做零售系统的公司，大客户提了个需求：要一个批量导入订单的按钮，因为每天早上要录几百条单子，太慢。

研发照着做了。做完客户又说不好用——导入模板格式太严格，稍有不对就失败，还得一条条改。改了三版，客户还是不满意。

后来有个人跑去客户那边看了一早上，发现问题根本不在导入按钮上：那些单子是从另一个系统里出来的，客户是先把那边导成 Excel，再在这边导进去。

真正的解法是跟那边的系统做个对接，订单自动过来。做完之后，那个按钮客户一次都没用过。

客户说“我要一个批量导入按钮”，他说的是他想到的解法。他真正的问题是“每天早上录单子要一小时”。

这就是 L4 那一层（懂需求源头）：分辨"他要的是这个"和"他要解决的是这个"。

还有一层更难：业务方的需求常常带着部门立场。销售提的需求会偏向成单方便，客服提的会偏向少挨骂，财务提的会偏向合规。都合理，但资源有限，你不可能全做。

这时候"懂业务"意味着你知道各方真正要的是什么，以及谁的那个更接近公司现在的目标。这是 L5。

所以别把"听业务的话"当成懂业务。真正懂业务的人，会听，但不会照单全收——他会追问一句：你想解决的是什么？不做这个会怎样？

七 三种"假懂业务"

业务理解有个麻烦：它很容易被自己误判。下面三种假象最常见，都能自查。

第一种，术语说得很溜。开会能接住业务方的词，听起来很懂行。但术语是外壳，说得"漏斗转化率"不等于知道他们的转化率为什么掉。检验很简单：让他讲一个具体的客户，讲不出来就是外壳。

第二种，把流程背下来了。知道一个单子走几步、每步谁批。这比第一种强，在 L2 到 L3 之间。但流程会改，而且改的原因通常不在流程里。真懂的人能说清"这一步当年为什么加"，背下来的人说不清。

第三种最隐蔽：跟业务方关系很好，但从不反驳。大家叫他"最懂业务的研发"，因为他跟销售称兄道弟、有求必应。问题是他从没说过"这个不该做"。

关系好是好事，能拿到别人拿不到的信息。但有求必应不等于懂，那只是服务态度。前面那句"一个从不说的人，谈不上懂业务"，放在这儿一样成立。

三种假象有个共同点：都能让你在会上显得很懂，但都经不起一句“那你觉得这个不该做吧”。

八 一个必须说清的边界

这一章全在讲业务，很容易被读成“技术不重要了”。

不是。这个误读比不懂业务更危险。

技术的价值不是没了，是转移了：从“会做”转移到了“知道能不能做、代价多大、风险在哪”。

小许做第一版的时候，技术上一点问题没有。他的问题在于不知道业务要什么。老谭能砍掉那个需求，是因为他既知道业务（做了也没人用），又知道技术（做这个要两个月）。两样都有，他才敢说那句话。

所以准确的说法是：

只懂技术不懂业务的人，会做出正确但没用的东西。只懂业务不懂技术的人，会拍脑袋做出做不到的决策。最稀缺的是两样都有的人。

这也回答了一个很多人问的问题：是不是得转产品、转管理才有出路？

不用。你可以一直在技术岗上，只要你往业务那一侧靠。实际上“懂业务的技术人”比“懂一点技术的产品”更稀缺，因为前者那半边更难补。

那些在这行走得远的人，大多数不是“从技术转行做管理”，而是在技术的底子上叠了一层业务。他们没放弃技术，是给技术加了一层。

九 三个信号，说明你开始懂了

业务理解是慢功夫，进步不容易自己察觉。有三个信号出现了，说明你确实在往里走。

第一，你能预判业务方的下一个需求。

不是猜中，是"他下次大概率会提这个"。能做到这一步，说明你已经理解了他面对的处境，而不只是他提过的需求。

第二，你能说出"这个做了，谁会不高兴"。

这是很有意思的一个指标。当你知道一个改动会让哪个部门不舒服、为什么，说明你已经看见了组织里那张利益关系的网。而这张网，从来不写在任何文档里。

第三，你开始被拉进非技术的会。

业务方讨论事情的时候开始叫上你，不是让你记需求，是想听听你的想法。

这三个信号一个比一个强。第一个出现，你在 L3 到 L4 之间；三个都出现，你已经靠近 L5 了。

十 入口很小，但要真的进去

业务理解不是看几篇文章能补的，它需要投入，而且是长期的。

但入口很小，三个就够：

- 多看一眼结果。你做的东西上线之后，谁在用、怎么用、骂了什么。绝大部分研发从来不看这个，做完就走
- 多问一句为什么。接需求的时候问"为什么要做、为谁做、不做会怎样"。大部分人从不问，因为问了显得事儿多
- 去一次现场。见客户、看工单、听客服录音、跟着销售跑一趟

具体怎么做、分人群怎么安排，第 11 章专门写。这里只把入口摆出来，因为这一章讲的是"为什么"。

小许那两周，成本不高。贵的是之前四年里，从来没人安排他去。

自检三问

- 1 你现在做的这个东西，最终用户是谁（不是产品经理，是真正用它的人）？你知道他拿它干嘛吗？
- 2 你负责的这块里，有没有一个"看起来不合理但其实有原因"的设计？你能说出原因吗？
- 3 上一次你说"这个不该做"，是什么时候？对方被你说服了吗？

可以做的事

个人

- 1 这周找一次你的功能的真实用户——销售、客服、客户都行——问三个问题：你什么时候用、用完还要做什么、最烦哪一步
- 2 挑一个你负责的模块，写一句话："它解决谁的什么问题"。写完念给一个非技术的同事听，看他懂不懂
- 3 下次评审会，试着问一次"不做会怎样"。问之前先自己想好答案

管理者

- 1 让研发去参加客户会议、看工单、听客服录音。这是成本最低的业务培训，没有之一
- 2 评审会加一个固定环节：先讲"这解决谁的什么问题"。讲不清楚的，不排期
- 3 把"砍对了什么"计进贡献。否则没人愿意当那个说"不"的人——说了要得罪人，不说也没损失
- 4 别只让架构师见客户。越是一线的人，越需要知道自己做的东西给谁用

别的岗位呢

这一章的道理对研发成立，对所有"做东西给别人用"的岗位都成立。

设计师不懂用户的使用场景，会做出好看但不顺手的東西。数据分析师不懂业务的决策链条，会做出正确但没人看的报表。法务不懂这单生意的来龙去脉，会给出严谨但谈不成的条款。

规律是同一条：你离最终使用者的距离，决定了你产出的价值。AI 能缩短"做"的距离，缩短不了"懂"的距离。

判断力

只能靠做决策、担后果来练

判断力不是想出来的，是做了决定、挨过后果之后长出来的。

这一章的三件事

- 判断力有三个成分，最容易被忽略的是第三个：承担
- 大部分人练不到它，不是因为不想，是因为没机会：而且学费没人替你交
- 一个残酷的地方：市场分不清"没做过"和"做不了"，所以你得留下证据
- **小孟押错了，又押了一次**

小孟工作第四年。他们组要做一套检索能力，搜商品也搜文档，老板的要求是"快、准、能模糊匹配"。

leader 在会上问谁跟一下。这次小孟举了手。

他花了两周：列了三个候选方案，各自搭了个最小验证，用真实数据跑了压测，最后写了一份十二页的对比报告。他选了B方案，理由是数据量翻十倍的情况下它的延迟还能稳住，而A方案在十万条以内更快，超过就掉。

上线两个月后出事了。有个大客户导入历史数据，一次灌进来三十万条，索引构建把内存吃满，服务挂了四十分钟。

回滚、扩容、重新导入。事故复盘会上，他说了三句话：

"是我选的。我压测的时候只测了增量导入，没测批量灌入。我以为不会有人一次灌三十万条，这个'以为'没有根据。"

leader 没换掉他，说了句"接着干"。

那两个月他不太好过。组里有人私下说他想出风头，还有人问他"至于吗，跟着 leader 走不就完了"。

第二年，同样的场景又来了一次，要做数据分析能力的选型。他又接了。

这次他做的第一件事不是搭环境，是去问业务方：峰值出现在什么时候？一次大概多少条？

答案是月底集中导入，一次几万到几十万。

他这次选了A方案，跟去年相反。理由写在报告第一页：我们的场景不是高频均匀增长，是月底集中灌入，所以批量构建能力比稳态延迟更重要。

这次没出事。

后来他跟我说，那次事故是他这几年最值钱的一件事。

"以前我觉得判断就是把事情想清楚。那次之后我才知道，判断是要押注的，押错了你得在场。"

二 判断力到底是什么

它不是聪明，也不是经验多。拆开看是三个成分。

模 型

判断力到底是什么

成分	是什么	怎么检验
信息不全时敢取舍	完备信息下的选择不叫判断，叫计算。判断就发生在信息不够、时间不够的时候	你能不能在只有七成信息的时候拍板，并说清缺的那三成是什么
对后果有预判	不是知道会怎样，是知道"可能出什么事、最坏到什么程度、我兜不兜得住"	决定做之前，你能不能说"如果出事，最可能是哪一步"
愿意承担	出事了算你的	复盘会上那句话是谁说的

第三条最容易被忽略，也最关键。

判断和建议的区别只有一条：出事了算谁的。

小孟如果只写报告不拍板，那份十二页的对比就只是一份建议。他之所以练到了东西，是因为他说了"我选 B"，然后半夜爬起来处理事故。

不承担后果的判断，不产生判断力。这一点怎么强调都不过分。

三 谨慎不是判断力

这里有个混淆，得拆开。

不少人觉得自己不拍板是"谨慎"，把谨慎当成负责任。两回事。

谨慎是信息不够的时候不做决定，等着。判断是信息不够的时候照样做一个决定，同时说清楚缺的是什么、什么情况下这个决定就错了。

前者很安全，也练不到任何东西。后者有风险，但只有它能长出判断力。

怎么分辨自己是哪种？看你会不会说这句："我现在只有七成信息，我按这七成定，缺的那三成是这几个，我盯着。"说得出来，是判断。说不出，只是在等。

还有一种更常见的伪装：把"我做完了，您看行吗"当成参与决策。那不是，那是把决定推回给别人，顺便给自己免了责。真正的参与是你有倾向，并且说得有理由。

四 为什么大部分人练不到

第3章讲过阿凯：不是不想，是没人让他想过。这里把原因说透，一共三条。

第一，没有机会。过去组织只把决策权发给架构师和总监。一个普通开发干到第十年，可能一次都没真正拍过板。他的每一天都是"需求已经定了，你来实现"。

第二，有机会也不敢接。接了就要担责。做好了未必有奖——因为"没出事"是看不见的；做错了一定有罚，因为出事是看得见的。这笔账大部分人算过，觉得不划算。

这个账其实算得没错，短期看确实不划算。问题是长期看，你是在把经验存进一个持续贬值的账户。

第三，组织也不真的鼓励。很多公司的容错是嘴上的。真出了事，该罚还是罚，该调岗还是调岗。一次之后，全组都学会了不接。

这里有个听起来有点冷的话得说：练判断力是要交学费的，而学费通常没人替你交。

小孟那两个月的低谷就是学费。老林那半年口碑下滑也是学费。

问题在于，很多人交不起：不是钱，是位置。家里有房贷、孩子刚上学、团队里正有人盯着你的坑，这时候让你主动去接一个可能出事的决定，是强人所难。

所以这件事不能只靠个人勇气。组织那一侧得动，下面第九节讲。

五 一个残酷的地方：市场分不清"没做过"和"做不了"

这一节是我特别想写的。

一个人十年没做过决定，和一个做过十次决定、错了一半的人，在简历上长得差不多。市场看不到"他没机会"，只看到"他没做过"。

阿凯就是这样。他不是做不了决定，是从来没人让他做过。但在面试官眼里，他和一个确实做不了决定的人没有区别。

这不公平，但改不了。能改的只有一件事：留下证据。

具体做法是：把你的判断写下来。

做决定的时候，不管多大，写三行：

1. 我判断什么（选 A 方案，理由是……）
2. 我认为会怎样（三个月内日均能撑到多少，超过就要扩容）
3. 什么情况说明我错了（如果峰值出现在集中导入，那我的假设不成立）

写完存着。三个月后、半年后回头看。

这个动作有两个好处。

一是复盘有依据。人的记忆会自动美化自己的判断——事后你多半会觉得自己"当时就想到了，只是没说"。白纸黑字能治这个。

二是这些记录本身就是证据。你需要它们来证明你做过判断。在晋升答辩上、在面试里，“我说过这句话、当时是这么想的、后来验证了一半对”比任何形容词都有力。

没做过和做不了是两回事，但只有写下来，别人才分得清。

六 怎么练

四条，从今天就能开始。

第一，从小决定起步，并且明确说“后果我担”。

不用等一个大项目。一个技术选型、一次排期取舍、一个“这两件事先做哪个”，都算。关键不在大小，在于你说了算，而且说了后果是你的。

小决定还有个好处：错了代价小，但练到的东西一样。

第二，把判断写下来。就是那三行。给个实际的例子，小孟第二年做选型时写的：

我判断：月底集中灌入是这个场景的主要形态，所以这次选 A，批量构建优先，稳态延迟往后放。我认为会怎样：单批 30 万条以内构建不超过 15 分钟，超过就要拆批。什么情况说明我错了：如果实际跑出来是高频小批量写入，A 的稳态延迟会拖垮查询，那就得换回 B。

三行，五分钟。半年后回看，要么证明对了，要么一眼看出错在哪一层。

第三，复盘要具体到层。别停在"我判断错了"。要分清错在哪一层：

- 是信息不够（压根不知道有批量导入这回事）
- 是权重错了（知道，但以为不重要）
- 还是根本没想到这个变量（脑子里没这个模型）

小孟那次是第一层。第二年他去问业务方，就是在补第一层。不同的错，补法不一样。

第四，找个人对练。找一个正在做判断的人，看他在会上是怎么想的、怎么说的、怎么收口的。这东西靠观察学比靠读书快得多。

还有一句：数量比大小重要。做十个小的判断，比憋一个大招有用。判断力是样本量堆出来的，不是靠一次豪赌。

七 判断力强的人长什么样

这个东西很虚，得落到能观察的行为上。我留意过一些在这件事上明显强的人，有三个共同点。

第一，他们先问"不做会怎样"。接到一件事，普通反应是想怎么实现，他们的第一反应是这事儿值不值得做。这句话问出去，一半的需求会自己缩水。

第二，他们给得出"错了会怎样"。不是"我认为这样更好"，而是"我选这个，如果出现 X 就说明我错了，到时候换成 Y"。给得出后半句的人，判断力通常不会太差，因为后半句要求他真的想过后果。

第三，他们改口很快。判断力强不等于每次都对。区别在于证据变了的时候他们改得干脆，不纠结于"我上礼拜刚说过"。死守上一个判断的人，其实是把面子看得比对错重。

反过来也有一条：判断力弱的人最常见的特征不是"想错"，是"不想"。会上不表态，会后提意见；出了问题能说出十个当初不该这么做的理由，但要拍板的那一刻一句都没有。

这三条都能观察。下次开会留意一下，谁在问第一句，谁在说第二句。

八 demo 到产品之间隔着什么

这两年到处都在发生同一个场景。一个人用工具，两三个小时做出一个能跑的东西：界面有、流程通、演示起来挺顺。兴冲冲拿去给 leader 看，得到的第一个问题常常是：上线之后怎么回滚？

一句话就把人问住了。做它的人自己也从没想过这件事。

这不是他偷懒，是这段路本来就不在那两个小时里。

差在哪儿

demo 回答的是"技术上能不能做通"。产品要回答的是另一件事：在现实里能不能长期成立。

前者只有一组约束，能跑就行。后者面对的约束完整得多，而且大半不在代码里。粗粗分三类。

第一类，真实环境。demo 跑在你自己的机器上，数据是干净的，请求一条一条来。真实环境里数据是脏的、量是突变的、依赖的接口会超时、半夜三点有人踩到你没想到的路径。工具不知道这些，它没在你的环境里待过。

第二类，出了事怎么办。上线意味着你得对后果负责：失败怎么退回、日志够不够半夜排查、有没有开关能先摁掉、会不会连带拖垮别的系统。demo 不需要这些，因为它不需要负责。

第三类，时间。产品是要活很久的东西。业务会改，需求会长，半年后接手的人可能不是你。所以得多想一层：这个结构有没有留缝，半年后加功能要不要推倒重来。demo 只活一次演示，没人回来改它。

所以 demo 到产品之间差的不是工作量，是一整套约束被拉进来之后的重做。而工具不知道你有什么约束，它只能按最通用的那种环境给你答案。

这才是"没有判断力做出来的永远是 demo"这句话的真正意思：不是说它跑不起来，是说它经不起被放进真实处境里。

这跟"研"和"发"是一回事

第 1 章拆过这两个字：发，是把已经确定的东西翻译成实现；研，是那些拿不准的部分——定向、取舍、试探。

demo 是"发"的产物。目标清楚、路径清楚、跑通就算完成。工具在这一步的效率提升最猛，因为它本来就是最标准、最可编程的那一截。

而从 demo 走到产品，每一步都是"研"。并发要不要现在解决还是先扛着；这个兼容层留一年还是立刻拆；先满足 A 客户的特殊用法还是做成通用的。这些答案取决于你处在什么处境，不取决于技术本身的优劣。

工具把"发"的成本压到接近零，顺带把"研"的比重推到了前所未有的高度。同样一件东西里，真正由你定的那部分，现在占了绝大部分。

第 4 章那个公式在这里才完整：同样用一款工具，产出能差三倍，差就差在这段没有标准答案的路怎么走。

落到日常：纠偏

具体到每天跟工具一起干活，这段判断最常见的样子是纠偏，三件事。

一是判断它说的的问题是不是问题。工具很擅长把风格问题讲成严重缺陷，也常常对一个真要命的地方只字不提。

二是判断它给的思路对不对。它经常选一条"能跑通但代价在后头"的路，因为代价不在这一轮里，它不知道你们下个季度要接什么。

三是判断还有没有更优解。它默认给的是最常见的方案，不是最适合你这个场景的方案，这两者经常不是一回事。

不纠偏会怎样？两个后果：代码变臃肿，方案变复杂。

工具的默认动作是"加东西"，加一层抽象、加一个兜底、加一个开关。它不知道你们系统里哪些东西早就没人用了，也不懂哪些边界碰不得。而复杂度只会往上长，没人管就一定失控。

一句糙话：AI 负责给选项，你负责决定哪个是真的、哪个是多余的。

所以架构师更重要了

顺着这条线往下，就到了架构师。

工具用得越多，架构师这个角色越重要。这件事我这两年越来越确定，而原因不在技术，在业务。

上面那三类约束里，最难、也最值钱的是第三类：时间。说白了就是"这个业务会长成什么样"。而回答这个问题的能力，恰恰是架构师的本职。

架构师做的事，往白了说是两头接：一头接着业务，知道这东西为什么做、明年大概往哪改、哪里能让步哪里不能；另一头接着技术结构，把这些取舍落成边界、层次和依赖。他给出来的从来不是"技术上最优"的方案，是"对这个业务、在这个时间点上最合适"的方案。

工具能给局部最优，给不了这个。它不知道你的业务三年会长什么样，所以它不知道哪里该留缝、哪里该堵死。

这就是驾驭这两个字的意思：不是在技术上比工具懂得多，是手里握着工具没有的那部分上下文——业务的、组织的、历史的——并且能用它做取舍。而这正是"研"的全部内容，也正好是工具填不上的那一段。

从这个角度看，AI 时代的架构师不是变清闲了，是变关键了。以前他要管的是结构不被写坏；现在他要管的是：在产出翻了好几倍的情况下，这一堆东西还是同一个系统的组成部分，并且始终朝着业务要去的方向长。

技术经验到底还有没有用

最后回应一句常见的担心：既然工具都能写了，攒了十年的技术经验是不是白攒了。

不是白攒，是换了个位置。过去它主要用来"写"，现在更多用来"验收和收口"：这段在你的环境里会不会出事，这个方案半年后有没有退路，加进来的这一层是不是多余的。会写确实不值钱了。能看不该这么写，从来没像现在这么值钱。

九 组织这一侧

个人能做的有限，因为机会在组织手里。三件事。

第一，把决策权下放。能下放又不会出大事的，尽快下放。这是团队唯一能练判断力的途径，别的招都没用。

怎么判断"不会出大事"? 看错了能不能回滚。能回滚的决定，就该放下去。

第二，把"追责"换成"复盘"。允许犯错不等于不管。区别在于出事之后的第一句话是"谁干的"还是"哪一步没想到"。

前者一次之后全组沉默，后者能让人把学费真正学到手。

第三，别让"做错了"和"不做"的代价倒挂。

这一条最要命。现在很多组织里的实际结构是：做了决定出了事，挨罚；不做决定，什么都不会发生。

在这个结构下，理性的人都不会做决定。不是他们没担当，是你把担当的价定得太高了。

最后说一句判断团队的话：一个从没人拍板的团队，不叫稳，叫脆。平时看不出来，因为大部分时候按流程走就行。等真需要有人站出来决定的那天，你会发现没人会。

而那一天一定会来。

自检三问

- 1 过去一年，你做过几个"后果算你的"决定？有没有一个？
- 2 你最近做的那个判断，能说清"什么情况说明我错了"吗？
- 3 在你的团队里，"做错了"和"没做"，哪个代价更大？
- 4 上一周，你驳回过工具给的哪段代码或哪个方案吗？驳回的理由，说得出来吗？

可以做的事

个人

- 1 这个季度主动接一件要拍板的事，写清楚"我判断什么、错了怎么办"，然后真的担下来
- 2 从今天起给判断留证据：三行，写完存着，三个月后回看
- 3 复盘一次你过去的错误，分清楚是信息不够、权重错了，还是压根没这个变量

管理者

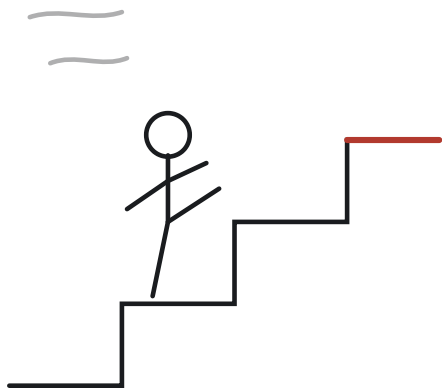
- 1 列一张清单：你手里握着的决定里，有哪些是"错了能回滚"的？这些全都可以下放
- 2 出事之后的第一句话，改成"哪一步没想到"。坚持三次，团队就敢接了
- 3 检查一下你们的奖惩结构。如果"不做"永远比"做错"划算，那没人会做决定，骂也没用
- 4 给主动接决定的人一点保护：出了事，只要复盘是诚实的，别急着换人

别的岗位呢

医生、律师、教师这些职业不太焦虑 35 岁线，不是因为他们更聪明，是因为他们的培养体系很早就强制要求做判断并承担责任。住院医要独立值夜班，年轻律师要独立见客户。他们从二十几岁就在往这个账户里存钱。

IT 行业的问题在于，我们设计了一套让人可以十年不做决定、还能拿到不错回报的体系。这是行业的锅，不完全是个人的。

好消息是，这套体系正在被打破——不是被人打破的，是被工具打破的。机会少了，但练判断力的成本也降了。



PART THREE

第 三 部 分

跃迁

能力怎么长出来

OPT

从技能互补到判断互补

OPT 不是一个人干所有活，是一个人具备一整个团队的建制。

这一章的三件事

- 这里的 T 是 team，所以重点从来不是人数，是建制
- 过去团队存在的理由是技能互补，AI 补齐技能之后，变成了判断互补
- 所以门槛不是"你会不会干活"，是"你能不能做出一个完整团队所需的全套判断"

— 阿岩的六周

阿岩 37 岁，去年从一家中型公司出来。走的时候没什么戏剧性，团队合并，名额有限，他排在了后面。

出来之后他接了个活：给一家做建材的贸易公司做一套内部系统，管进货、库存、报价、对账。客户不大，预算不多，按人头算，这种项目过去至少要三四个人干两三个月。

他一个人接了。

技术上他并不全会。前端他这些年写得少，部署和运维他懂个大概，测试更是从来没专职做过。但他有六周时间和一堆工具。

六周后他交付了。不是演示版，是客户真的在用——二十几号人每天登录，跑到现在快一年。

我问过他这六周怎么过的，他说的和我想的不太一样。

我以为重点是他怎么用工具补齐了不会的技能——前端不会写就让工具写，运维不熟就让工具教。这些他都做了，他自己说"那部分其实没想象中难，问两句就会了"。

真正难的是别的。

第一周他基本没写代码。他泡在客户那边看他们怎么干活：进货单是谁填的、报价是怎么算出来的、对账为什么总要拖三天。他发现客户真正卡住的不是"没有系统"，是报价规则散落在几个人的 Excel 里，每次报价都要翻。

于是他把第一个版本砍了一半——原本要做库存预警、供应商评级、移动端，他只留下了进货、报价、对账三件事。客户不太高兴，他解释了两次。

第三周他推翻了一次自己的设计。他原本做成了"所有人共用一个流程"，跑起来才发现仓管和销售的用法完全不同，硬凑到一起两边都别扭。改成两套视图之后才顺。

第五周他做了一件当时看起来最没用的事。他写了一份二十来页的东西，讲这个系统怎么用、数据错了怎么改、出了问题先查哪里。客户那边没人要求他写。

那件事后来救了他。系统上线第二个月，客户有人误操作把一批进货单删了，打电话过来。阿岩在电话里照着那份文档，让对方自己恢复了。

他说了一句我觉得特别有意思的话：

"以前我以为一个人干不了大活，是因为活太多。这六周我发现不是。是因为要做的判断太多。"

技术不是障碍，那部分工具替他扛了。难的是：做什么不做什么、做成什么样算好、什么时候该推翻重来、出了问题怎么办。

这些过去是分给不同人的——产品经理决定做什么，架构师决定怎么做，测试决定什么算合格，项目经理决定什么时候停。现在全在他一个人身上。

这就是 OPT 真正的门槛。

二 T 是 team，所以重点不是人数

现在市面上讲"一人公司""超级个体"的很多，大部分讲歪了。

它们给人的画面是：一个人，关在屋里，靠 AI 干出一个团队的活。听起来很酷，也让人很焦虑，因为你会觉得"我一个人可干不了那么多"。

但 OPT 里的 T 是 team，指的是建制，不是人数。

它的意思不是"一个人把所有人的活都干了"，而是"一个人（或者极小的一个单元）具备一个完整团队该有的全套功能"。

这两句话差别很大。前者是体力问题，后者是能力结构问题。

而且它也不排斥多人。三个人组成的 OPT，和一个人组成的 OPT，性质是一样的：只要这个单元内部具备完整的建制，不需要依赖外部来补位。

所以别被"一人"这个词带偏。你要问自己的不是"我能不能一个人干完"，而是：

如果只给我和我手上的工具，我能不能从"有人提了个想法"走到"用户真的用上了并且没出事"？

这条路走得通，你就是个 OPT，不管你是一个人还是三个人。

三 团队存在的理由变了

理解 OPT 的关键，是想清楚一件事：过去团队为什么必须存在？

答案是技能互补。

你要做一件完整的事，需要前端、后端、测试、运维、产品、设计。一个人不可能全会，所以只能凑人。团队的最小规模，由"凑齐这些技能需要几个人"决定。

这也是为什么过去"全栈"这个词含金量那么高——因为它意味着你可以少凑一个人。

现在这个约束松动了。工具把技能补齐这件事变得很便宜：你不会前端，它能写；你不会运维，它能教；你不会写测试，它能生成。

技能不再是门槛之后，团队存在的理由就变了——从技能互补，变成判断互补。

一个人加工具，技能上已经接近自足。真正撑不住的，是判断的量：

模 型

团队存在的理由变了

一个项目要做的判断	过去谁负责
做什么、不做什么	产品、业务负责人
做成什么样算好	产品、设计、验收
技术方案怎么选、代价多大	架构师
什么情况算做完了	测试
什么时候该停、该止损	项目经理
出事了怎么办、谁兜	负责人

这张表就是"建制"的具体含义。

一个 OPT 不是要自己干完这张表里所有人干的活，那大部分是执行，工具能扛。他要的是能对这六行都做出判断，并且对结果负责。

所以门槛可以讲得更直白一点：

不是"你会不会干活"，是"你能不能做出一个完整团队所需的全套判断"。

阿岩那六周，卡住他的从来不是前端写不写得出来。

四 三个衍生变化

这个转变会带来几个不那么显眼、但影响很深的变化。

第一，"全栈"换了含义。

过去全栈 = 会前端也会后端，是横向的技能覆盖。

现在全栈 = 能从需求走到交付，全程负责。技能这一层可以靠工具补齐，所以技能覆盖不再稀缺；稀缺的是全程不甩锅。

你会发现一个现象：现在真正能一个人扛事的人，未必是技能最全的那个，而是从不含糊"这算不算做完了"的那个。

第二，协作对象变了。

过去你大量时间花在跟人协作上：对齐、确认、解释、催进度。这些协调成本很高，因为人和人之间的对齐天然困难。

现在相当一部分协作对象变成了 Agent。跟 Agent 对齐的成本趋近于零：它不需要开会，不会误解你的情绪，你说清楚它就做。

所以"沟通协调"的重要性在下降，而"任务拆解"和"验收标准"的重要性在上升。

过去你跟人说不清楚，对方会追问；现在你说不清楚，它就按它理解的做，而且做得很自信。所以拆解和验收这两件事从"加分项"变成了"核心能力"。这一条第 9 章展开。

第三，最稀缺的变成了"知道要做什么"。

执行便宜了，验证便宜了，那么瓶颈就上移到了最前端：做什么。

过去一个团队里，"决定做什么"只是其中一个角色的工作。现在它成了所有工作的起点，也是唯一不能被工具替代的起点。

这也是为什么前几章反复讲业务理解——它不是锦上添花，它是 OPT 时代的入场券。

五 也得说清楚 OPT 的代价

这一章不能只讲好的。

第一，OPT 不是谁都能做，也不是谁都该做。

它需要全套判断，而判断力是慢功夫（第 6 章）。判断力还没攒够的人去做 OPT，结果不是自由，是风险全落在自己身上还没人兜。

第二，OPT 意味着没有缓冲。

在团队里，你做错了有 leader 顶着，需求变了有人一起扛，出了事有流程。一个人做的时候，所有这些都落在你自己身上。阿岩那个项目第三周推翻自己的设计，那两周是纯损失，没人替他分担。

第三，自由是有价格的。

不用开会、不用对齐、不用汇报，这很爽。但代价是没人给你兜底、没人给你机会、也没人在你判断错的时候替你说句话。

所以别把 OPT 浪漫化。它是一条真实可行的路，但它不是逃避组织问题的避难所，它要求的东西比在组织里更多，不是更少。

顺便交代一句：我自己没完整走过这条路，这些年一直在组织里。上面这些是从别人的经历里看的，可能有偏差，当参考，别当样板。

对大部分人来说，更现实的路径不是“出去做 OPT”，而是在组织内部用 OPT 的方式工作：主动把一个完整的事端到端扛下来。这样你有组织的兜底，同时练到全套判断。

自检三问

- 1 如果只给你和工具，从"有人提了个想法"到"用户真的用上了"，你能走完吗？中间哪一环你会卡住？
- 2 上面那张六行的判断表，你能对几行做出判断？哪一行你从来没想过？
- 3 你现在做的事，是"团队里的一环"，还是"一件完整的事"？

可以做的事

个人

- 1 挑一件小事端到端做完：从理解需求，到上线，到看数据，到处理反馈。中间不甩给任何人
- 2 做一次"建制盘点"：拿上面那张六行表，看看哪几行你从来没做过判断。挑一行，这季度补上
- 3 别急着攒技能。技能这层工具能补，判断这层补不了

管理者

- 1 团队规模可以缩小，但别只缩规模。人少了，每个人要做的判断会变多，得同步把决策权给出去
- 2 试着把一个完整的小项目交给一个人端到端负责，而不是拆给五个人各做一段。这是练 OPT 最好的方式
- 3 别再用"人头"评估产能了。同样的活，一个 OPT 加工具可能比五个人快，也可能更慢——取决于他的判断力，不取决于人数

别的岗位呢

OPT 不是程序员的专利。

一个独立设计师加工具，可以完成过去一个小组的活：从需求沟通到提案到交付到改稿。一个独立咨询顾问可以一个人做完过去两三个人的项目——调研、分析、报告。一个自媒体创作者更是早就 OPT 了：选题、写作、剪辑、运营、商务，一个人加一堆工具。

规律一样：技能被工具补齐之后，能不能独立，取决于你能不能做出全套判断，以及敢不敢对结果负责。

五能阶梯

能做、能写、能说、能讲、能聊

每一层都在扩大你的影响半径。而 AI 时代，是半径越大越值钱。

这一章的三件事

- 五能是一级一级往外扩的影响半径，不是五种并列的能力
- 各层在 AI 时代的涨跌不一样：能做在跌，能聊在猛涨
- 还有一条最容易被忽略的：对 AI 的表达，和对人的表达，是同一种能力

— 老范

老范技术很好。他是我们这行里那种典型的“能人”：系统出疑难问题别人查不出来，他上；性能卡在瓶颈上谁都没辙，他能调；新人看不懂的老模块，只有他讲得清。

他在一家公司待了九年。九年里换过三任 leader，每一任都认可他。

但每一次晋升都没有他。

第一次他以为是年限不够。第二次他觉得是名额问题。第三次，新来的 leader 提了个比他晚来四年的同事，他去找 leader 谈。

leader 说的话挺实在，我转述一下大意：

"你的活儿我完全放心，交给你的事从来不用盯。但我没法让你带更大的范围。因为我不知道你怎么想的——你做完就交了，中间不出声，会上也不怎么说话。我要往上提一个人，得知道他能不能带着一群人把事想清楚。我不知道。"

老范后来跟我说，他当时第一反应是委屈：我活干得比谁都好，为什么不行？

他不知道的是，"活干得好"影响到的只有他自己那一份。而"带更大的范围"，需要的是让更多人知道你怎么想、愿意跟着你走。

这两件事的能力结构完全不同。

我认识很多老范这样的人。他们卡住的位置都一样：能力很强，半径很小。

二 五能是一级一级往外扩的

有个说法我一直觉得挺好用，很简单，五层：

能做、能写、能说、能讲、能聊。

这五个不是并列的五种能力，是一条阶梯——每一层都在扩大你的影响半径。

模 型

五能是一级一级往外扩的

层级	影响半径	你能撬动什么
能做	你自己	你产出的东西
能写	能读到你文字的人	跨时间、跨空间的影响
能说	对面的人	即时的、可交互的影响
能讲	一群人	一对多的动员和说服
能聊	任何背景的人	跨领域的影响力和信任

半径越大，能撬动的东西越多，价值也就越大。

而 AI 时代的变化，正是"自己能做"这一层在贬值，而"能撬动多少人"在升值。所以五能越高层越值钱，这不是巧合，是同一件事的两种说法。

具体怎么看这五层在 AI 时代的涨跌：

模 型

五能是一级一级往外扩的

层 级	AI 时代的 变化	说明
能做	跌	这一层正在被工具直接接管，没有悬念
能写	分层	"写的手艺"在跌（工具写得比多数人顺）， 但"写什么、写给谁、想达到什么效果"在涨
能说	涨	即时的、带语境的沟通，工具替代不了
能讲	涨	一对多需要结构和感染力，工具能给稿子给不了场
能聊	涨得最猛	跨领域、靠共情和信任推进，机器完全没有机会

"能写"这一层值得单独说一句，因为它被误解最深。

现在很多人说"写还用学吗，工具两分钟一篇"。确实，工具能把一段话写得通顺、完整、挑不出毛病。

但它不知道你要达到什么效果。

一份事故复盘，写给老板看和写给团队看，重点完全不同：老板要看的是"影响多大、怎么收尾、还会不会再出"，团队要看的是"哪一步没想到、下次怎么发现"。同样的素材，不同的目的，写出来是两份东西。

工具解决的是"写得通顺"，你解决的是"写这个是为了什么"。后者是判断，不是手艺。

所以“能写”这门功夫贬值的是手艺那半边，升值的是判断那半边。

三 一个漂亮的对称

这里有个我觉得挺有意思的对称。

第 1 章讲，AI 吃掉了“把需求翻译成代码”——把人的话翻成机器的话。

但与此同时，“把专业翻译成人话”反而变得更值钱了——把机器的、技术的、专业的东西，翻成别人能懂、能信、能据此做决定的话。

沟通本质上也是一种翻译，只是方向相反。

技术方向的翻译被机器接管了，人的方向的翻译就成了稀缺品。

这也解释了为什么老范这类人会卡住：他最强的那项是“把人的话翻成机器能执行的东西”，而这项正在被接管；他最弱的那些项，恰好是正在升值的那些。

四 对 AI 的表达，也是表达

这一节可能是这一章里最被低估的。

我听过一个说法：现在提示词工程的门槛降了，不用再琢磨怎么问了，所以表达不重要了。

不对。

门槛降了是真的，但不同的人用同一个模型，结果的差距依然很大，这个现象并没有消失。原因很简单：

AI 是“根据你说的推断你想要的”。你说得越清楚，它推断得越准。

你跟它说"帮我写个导出功能", 它会给你一个导出功能。你跟它说"给销售用的, 他们每周一早上要发给客户, 数据量大概两千行, 字段要跟 CRM 里的一致, 导出来之后不改就能发", 它给你的东西完全是另一个水平。

差别不在措辞技巧, 在于你把背景、约束和目的说清楚了没有。

而这跟对人的表达, 是同一种能力——把脑子里的东西结构化地说清楚。

所以那个公式要再升一级:

过去, 沟通能力 = 你与人协作的效率。现在, 沟通能力 = 你与 AI 之间的带宽。

它是"杠杆的杠杆"。你的判断力再好, 如果说不清楚, 传给人的时候打折一次, 传给 AI 的时候再打折一次。

反过来, 带宽大的人, 同样一份判断能撬动的人和机器都更多。

五 为什么"能聊"对技术人员最难

五层里最难的不是"能讲", 是"能聊"。

讲是有准备的、有结构的、你掌握主动权的。聊不是——聊是非确定性的、跨领域的、要靠共情往前推的。

技术人员难在三个地方。

第一, 习惯确定性语境。

我们这行的日常工作都在确定性很高的语境里: 代码、协议、明确的需求。对就是对, 错就是错, 一句话只有一个意思。

而"聊"发生在模糊语境里。对方说的话往往不准确, 需要你猜他的意思, 还要接得住。这两种模式几乎不兼容。

第二，"纠正欲"。

这大概是技术人员最常见的社交障碍。听到一个不准确的表述，第一反应是想纠正它。

在代码评审里这是美德。在饭桌上这是社交杀手。

我见过一次典型的故事：一次聚餐，有人聊起某个热点新闻，说了句"听说那个系统是黑客攻击"。在座一位工程师立刻纠正："那不算黑客攻击，那是内部人员泄露，性质不一样。"

他说的是对的。那顿饭之后大家聊天的氛围就变了，后面半小时没人再接那个话题。

他后来挺委屈：我说错了吗？

没没错。但那个场合里，正确不是最重要的东西。

第三，也是最难的一：知识面不够宽，或者不够好奇。

"能聊"要求你能跟不同背景的人找到共同语言。这需要知识面的广度，也需要对他人的真实好奇——你得真的想知道他在做什么、他关心什么。

知识面可以补，好奇心补不了。装出来的好奇一眼就能看穿。

小邱那次事故之后，我跟他说过一句话，也送给读者：在不需要正确的场合，别着急正确。

六 怎么往上走

五层是阶梯，得一级一级上，但起步动作都很小。

从"能做"到"能写"：先写给自己看。每个决定写三行（第 6 章那条），每件事做完写一段复盘。写完不用给任何人看，先练"把想法变成文字"这件事本身。

从"能写"到"能说": 找一个人讲一遍你刚做完的事, 看他能不能听懂。听不懂就是没说清楚, 不是他笨。

从"能说"到"能讲": 在组会上讲一次你做的东西。注意, 不是讲"我做了什么", 是讲"我为什么这么做、中间纠结过什么"。前者没人爱听, 后者才有价值。

从"能讲"到"能聊": 参加一次跟你工作无关的饭局, 全程只问别人, 不讲自己。这个练习听起来很简单, 做到的人很少。

还有一个通用的办法: 找一个你佩服的人在会上怎么说话, 把它拆开看。他先讲什么、怎么收口、什么时候停下来让人提问。这东西靠观察学比靠上课快。

自检三问

- 1 你的五能停在哪一层? 上一层你做过吗?
- 2 你上次做的一件事, 有多少人知道你是怎么想的?
- 3 你跟 AI 说一件事的时候, 有没有把背景、约束和目的讲清楚?

可以做的事

个人

- 1 这周写一份东西——复盘、方案、或者只是"我为什么这么做"。不用给任何人看，先写
- 2 下次在会上发言，试着讲"我纠结过什么"，而不是"我做了什么"
- 3 参加一次非工作场合的聊天，全程只提问、不纠正。回来看看自己能不能忍住

管理者

- 1 提人的时候，别只看"活干得好不好"。加一条：他能不能把想法说清楚、让一群人跟上
- 2 给团队创造"讲"的场合。组会不是进度同步会，让人讲讲"为什么这么做"
- 3 注意那些从不发言但活干得很好的人（老范型）。主动找他聊一次，告诉他你看到了什么：这种人往往不知道自己卡在哪
- 4 你自己也练。管理者最容易卡在"能讲"到"能聊"之间：会开会，但不会跟不同背景的人打交道

别的岗位呢

这条阶梯跟技术没关系。

会计做到"能讲"，才能从记账走到参与经营决策；设计师做到"能聊"，才能从接需求走到定义需求；销售做到"能写"，才能把一次成交变成一套可复制的方法。

规律是同一条：你能影响多少人，决定你能值多少钱。工具把"做"的成本降到了极低，于是"影响"成了新的稀缺品。

驾驭者

唯一一条年轻人不吃亏的赛道

这是 AI 时代新长出来的一个生态位。过去根本不存在这个位置，所以所有人站在同一条起跑线上。

这一章的三件事

- 五个去向里，前四个都吃资历，只有这一条不吃
- 它要求的新能力是任务拆解和验收标准，不是写代码
- 但别误读：它依然需要你懂那件事本身，技术不是门槛不等于门槛消失了

— 小韩的三天

小韩刚入职一年，做的是数据相关的工作。

去年他们组接了一批活：把过去几年积累的历史数据清洗一遍，统一格式，补齐缺失字段，导进新的库里。数据量不小，几十万条，来源五花八门，格式混乱。

组里先安排了老吴做。老吴工作八年，做事扎实。他的做法是自己写脚本：读文件、正则匹配、逐字段处理、遇到异常记日志。写了一周，跑通了大概六成，剩下四成是各种乱七八糟的情况——有字段缺失的、有编码混乱的、有重复记录的。

他打算再用一周把异常情况一个个处理掉。

小韩那阵子手上的活不多，组长让她也试试，“看能不能帮上忙”。

她没写清洗脚本。

她做了一件不一样的事：先花了半天，把那批数据的异常情况分了类——缺字段的、格式不对的、重复的、明显是脏数据的。分完发现，异常总共就那么六七种，真正难处理的只有两种。

然后她设计了一套流程：让 Agent 按批跑，每批跑完自动抽检5%，人工看一眼；不合格的批次自动重跑，重跑三次还不行就单独拎出来；最后输出一份差异报告，说清楚哪些改了、哪些没动、为什么。

三天出结果。覆盖率比老吴那一周高，而且后面又来了两批数据，直接跑同一套流程就行。

老吴看完说了句挺实在的话：“我一直在想怎么把数据洗干净，她想的是怎么让这事不用人一直盯着。”

后来这套流程被别的组拿去改了改接着用。

小韩不是技术比老吴强。她强在两件事：把活拆得清楚，把“什么算做完”定得明白。

二 第五个去向

前面几章陆续提到过人可以往哪走，这里攒齐了说：

- 向上：做取舍、担后果（第 10 章）
- 向下沉：端到端自己交付（第 7 章）
- 向深：做领域专家，攒 AI 拿不走的 know-how（第 5 章）
- 向外：做连接者，跨领域整合、懂业务和商业（第 5、8 章）
- 向 AI：做驾驭者——设计 workflow、定义 Agent 的分工、定验收标准、处理兜底

前四条有个共同点：都吃资历。

向上要判断力，判断力要时间攒。向下要全套能力，也得练。向深要领域积累，更急不得。向外要人脉和视野，也是慢功夫。

所以年轻人在这四条上都不占便宜，这是正常的，也是公平的。

只有第五条不一样。

因为它要求的東西，去年才成型。

在座所有人——刚毕业的、干了十年的、干了二十年的——在“怎么给 Agent 分活、怎么定验收标准、怎么设计兜底”这件事上，经验差距接近于零。没有人有五年以上的经验，因为这东西出现还没那么久。

这是我在这本书里唯一能给出的“年轻人不吃亏”的方向。如果你刚入行，或者正觉得自己资历浅、拼不过老的人，这一章是写给你的。

三 这是一个新增的生态位

要说清楚它为什么特殊，得先说清楚它新在哪。

过去一个团队里的角色是这么分的：产品经理决定做什么，研发写代码，测试验收，运维部署。每个人负责一段。

现在多出来一段，而且这段过去根本不存在：编排。

就是决定"这件事交给谁（人还是 Agent）、怎么拆、怎么验、不合格怎么办、出问题怎么兜"。

过去不需要这个角色，因为活都是人干的，人自己会判断进度和质量。现在相当一部分活是机器干的，机器不会自己判断——你得替它判断，还得替它设计流程。

于是就有了这个新位置：驾驭者。

它不等于"会用 AI"。会用是最基础的，人人都会。驾驭者是设计者：

- 决定一件事拆成几步、每步交给谁
- 定义每步的验收标准：什么算做对了
- 设计异常路径——做错了怎么办、谁兜
- 判断哪些环节不能交给机器

这四件事，跟会不会写代码关系不大，跟"你懂不懂这件事该怎么做"关系很大。

四 三条新能力

第一，任务拆解。

这件事过去也重要，但现在重要性提高了一个量级。

原因在第 7 章讲过：跟人协作的时候说不清楚，对方会追问；跟 Agent 协作的时候说不清楚，它就按它理解的做，而且做得很自信。

所以拆解的要求变了：不是"大致说清楚"，是边界清晰、输入输出明确、可验证。

小韩那半天做的分类，本质就是拆解——她把"清洗数据"这件模糊的事，拆成了六七种可分别处理的情况。拆完之后，每一种是能交给机器做的还是必须人来看，一目了然。

第二，验收标准。

这一条是最稀缺的，也是最容易被忽略的。

大部分人做事情，从来不定"什么算做完了"。他们是做着做着，觉得差不多了就交了。

跟人协作的时候这问题不大，因为双方会反复对齐。跟机器协作的时候不行——你不给它标准，它就自己定义"差不多"。

小韩那个流程里最值钱的一句不是"让 Agent 跑"，是"每批抽检 5%，不合格自动重跑，重跑三次还不行就单独拎出来"。这就是验收标准和兜底路径。

怎么练验收标准？有个笨办法很管用：做一件事之前，先写下"做完之后我怎么知道它做对了"。写不出来，说明你还没想清楚。

第三，异常处理的设计。

新手和老手在这件事上差别最大。

新手设计流程，默认一切顺利，跑通就完事。老手设计流程，先想哪里会出问题、出了问题往哪走。

Agent 有个特点：它出错的时候不会告诉你它错了，它会给你一个看起来很合理的错误结果。所以兜底设计不是可选项。

小韩那个"重跑三次还不行就单独拎出来"，就是兜底。

五 但别误读：门槛不是消失了，是换了

这一章很容易被读成"不会技术也能做驾驭者"。

不是。

小韩能设计出那套流程，是因为她懂那批数据——她知道常见的异常有哪几种、哪些是真脏数据哪些是格式问题、抽检 5% 够不够。这些判断是建立在她对这事的理解上的。

一个完全不懂数据清洗的人，设计出来的流程会是错的，而且错得很流畅——因为 Agent 会很高效地执行一个错误的方案。

所以准确的说法是：

技术不再是这条赛道的门槛，但判断依然是。

换了个门槛而已，不是没有门槛。好消息是这个新门槛不吃资历，它吃的是"你把一件事想清楚的能力"，而这东西年轻人一样能练。

而且这条赛道还有个隐藏好处：它反过来逼着你练判断力。

设计工作流这件事，本质上就是在做判断——拆得对不对、标准定得合不合适、兜底够不够。每做一次，你就练一次。这比等一个"拍板的大项目"要快得多，也便宜得多。

所以对年轻人来说，这条路的真正价值不只是"能干活"，而是它是一条练判断力的捷径。你在练驾驭的同时，括号里那两项也在长（第4章那个公式）。

六 给年长的人一句

这一章不能只写给年轻人。

如果你已经干了十年、十五年，看到这里可能会有点不舒服——好不容易攒的资历，在一条新赛道上清零了。

我的建议不是让你去跟年轻人抢这条赛道，而是：别把这件事外包出去。

很多资深的人现在的心态是"这些新东西让年轻人去搞，我管好业务就行"。这个态度有风险。

不是因为你要自己去写工作流，而是因为编排方式会反过来决定业务能做成什么样。哪些环节自动化、哪些必须留人、质量标准怎么定：这些是业务决策，不是技术细节。

不懂编排的人做这些决策，会做出保守的或者离谱的判断。

所以你不需要成为最会设计工作流的人，但你需要看得懂、判断得出好坏。就像你不需要会写代码，但你得看得懂方案。

自检三问

- 1 你最近做的一件事，有没有先定"什么算做完了"？还是做到哪算哪？
- 2 你交给工具的一件事，如果它做错了，你怎么发现？有没有兜底路径？
- 3 你有没有设计过一个能重复用的流程，而不只是完成了一次任务？

可以做的事

个人

- 1 挑一件你现在手上的活，试着把它拆成“能交给机器”和“必须人来看”两部分。拆完你会发现很多事根本不用自己盯
- 2 做一件事之前，先写一句“做完之后我怎么知道它做对了”。写不出来就先别动手
- 3 把你做成的一件事流程化——写下来，下次能直接用，也能给别人用

管理者

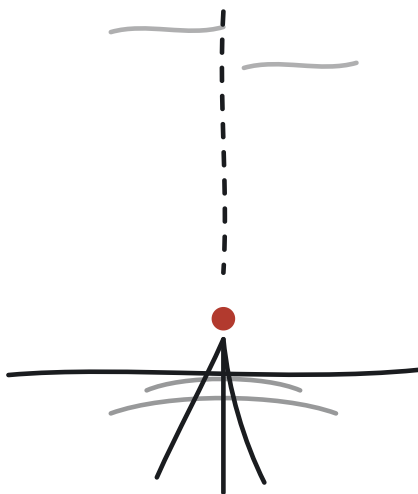
- 1 留意那些会设计流程的人，不一定是资深的那个。把这类人放到能影响更多事的位置上
- 2 别只考核“完成了多少”，加一条“有没有做成可复用的流程”。后者才是真正的产能
- 3 给年轻人机会做编排。这条赛道上资历没有优势，你按资历排的话，会错过最适合的人
- 4 你自己至少要能看懂 workflow 设计，判断得出好坏。这是新的必备素养，不是技术细节

别的岗位呢

驾驭者这个生态位不是技术岗专属。

市场部里设计内容生产流程的人、财务部里设计自动对账流程的人、法务部里设计合同初审流程的人——他们做的都是同一件事：拆分、定标准、设兜底。

哪个岗位先出现这样的人，哪个岗位的产能就会跟其他岗位拉开差距。而且这个差距会自我强化——流程越好用，越多人用，反馈越多，流程越好。



PART FOUR

第 四 部 分

落地

个人与组织各自怎么办

小 leader 的两难 向上，还是向下沉

AI 时代最先被掏空的不是一线干活的人，是中间那一层。因为"分解"本身，也是一种翻译。

这一章的三件事

- 你最熟练的那几件事，正在一起失效
- 组长真正值钱的不是拆得细，是敢砍什么，砍错了算你的
- 还剩两条路：往上，或者沉下去。原地最危险

那天早上，产品在企业微信上给老陈发了个文档：《Q3 客户管理模块需求》，后面跟一句：

"老陈，任务清单我让工具先跑了一版，你们看看，没问题直接分。"

他点开。二十六个任务，每个都带着预估工时、前置依赖、建议顺序，连"这两个可以并行""那个要等接口定稿"都标了。

他本来准备花两个小时干这件事。实际花了十三分钟：从头到尾看一遍，把两个任务的工时从 3 天改成 4 天——他知道那块代码有多烂，工具不知道——然后在群里回一句"收到，没问题"。

那天上午剩下的时间很难熬。他打开一个 bug，改了二十分钟发现根本不是问题，又关了。十点半的时候他想问问组里有没有人需要帮忙，话到嘴边咽回去了。问这个，显得自己很闲。

那种感觉不是轻松，是空。你练了四年的手艺，有天早上发现机器干得比你，而你还得装作没事。

后面两个月，这类事一件接一件。

周三那个雷打不动的对齐会取消了。管理层说看板上有数据，谁卡在哪、卡了几天，系统自动出摘要，不用人念。老陈当时还挺高兴，少开一个会。过了些日子他才反应过来，那是他每周唯一一次所有人都得听他说话的场合。

周报也不用手写了，系统从提交记录里生成，写得还挺像样。

他并没有闲下来。组里的人用了 AI 之后产出快了，需求来得更密，他要协调的事反而更多，还是每天最后一个走。但你若问他这周到底干了什么，他会愣一下，然后说一堆"推了一下那个""对齐了一下这个"，说完自己都觉得那不像个答案。

真正的转折在季度规划会上。

投屏上是下季度的候选需求。轮到他的时候，上面有人问："老陈，你们组就三件事，你觉得该做哪三件？"

他张了张嘴。脑子里过了好几个候选，每个都有道理：客户提的那个肯定要做，老板上次在会上点过名的那个不能放，还有一笔技术债，不做的后面越来越难改。三件事，四个候选，他排不出来。

"这个……得看产品那边怎么排吧。"

没人接话，投屏上的光标一直在闪。

散会以后他在工位坐到七点多，什么也没干。后来跟同学吃饭喝了点酒，说了句挺实在的话：

"我这四年，好像一次都没做过选择。"

老陈不是混日子的人。组带得不错，需求拆得清楚，谁卡住了他上手就能救，人也留得住。问题不在他不努力，也不在他能力差。

问题在于，他用四年练成了一件挺值钱的本事，而这件本事正被一台机器接走。

而且接走的方式特别安静。没人通知他"以后不用你拆了"。需求还在拆，进度还在同步，代码还在写，只是每一件都不再需要他。

这种事没有通知单。等你回过味儿来，通常两三年已经过去了。

二 把他的一周摊开看看

做个有点残忍的练习：把老陈每周做的事列出来，一项一项问——这事儿 AI 能不能干，干完了还有没有人需要他。

模 型

把他的一周摊开看看

他每周在做的事	大概耗时	说白了是干嘛	AI 时代的变化
拆需求、估工时、派活	8 - 10 小时	把目标翻成任务	降得最快，工具拆得比他细
盯进度、拉对齐	5 - 6 小时	传递信息	降得很快，看板加摘要就够了
向上汇报	2 - 3 小时	把下面的事翻成上面的话	降，系统自己会生成
救火写代码	8 - 12 小时	执行	降，而且不用等他有空
技术选型、方案评审	2 - 3 小时	判断	涨，可惜次数太少
判断哪个需求不该做	几乎没有	判断加担责	涨得最猛，这是真价值
为结果负责	有，但被动	担责	涨

最后两行最有意思：他花时间最多的四件事，正好是贬值最快的四件事；最值钱的那两件，他几乎没做过。

一句话说完：忙的量，和价值的量，脱钩了。

周三那个会被取消，是最典型的信号。组织需要中层，很大一个原因是信息不对称——上面说的底下不知道，底下干的上面看不见，中间那层就是根管子。现在看板、文档、群聊、自动摘要，所有人看到的都是同一份东西。

管子值多少钱，取决于两头堵不堵。两头一通，管子就归零。

三 拆任务，也是一种翻译

前面讲过，这行一直在干的一件事，是把人的想法翻译成机器能执行的东西。需求翻成设计，设计翻成代码，代码翻成部署。

补一句：把一个季度目标翻成二十六个任务，也是翻译。

它有翻译的全部特征：输入是确定的（目标别人已经定了）、输出能列全（任务、工时、依赖）、做完能验收（拆得全不全、估得准不准）。这三条，恰好是 AI 最擅长的三条。

剩下的才是“拆得对”。全不全、细不细，AI 已经做得比人好，下面这几件它干不了：

- 优先级。资源永远不够，先做哪个、放弃哪个，这是取舍，不是排队
- 砍掉什么。得知道做了会怎样、不做会怎样
- 风险谁扛。这个方案可能出事，出了事算谁的
- 什么时候停。继续投，还是止损

这几件事有个共同点：对不对，得等后果出来才知道。而 AI 不承担后果。

还有个坑。工具拆出来的清单常常看起来特别正确——全面、均衡、颗粒度一致。这最危险：它平均，所以没重点；它全面，所以不取舍。

他们组上个月就碰到一次。有个需求是给老客户加个导出功能，说实话一年用不了两回。工具照样把它拆成八个漂亮的任务，工时估得明明白白。没人觉得哪里不对，因为拆得确实没问题。

一个从不砍东西的分解，不是分解，是放大。而放大需求，恰好是组织最不需要的能力。

所以中间层的分水岭在这儿——值钱的不是"能分解"，是"分解得对"。而"对"的代价是，你得敢拍板，拍错了算你的。

顺带回答了另一个问题：判断力为什么这么稀缺。因为它只能靠做决策、担后果练出来，而这种机会，过去组织只发给架构师和总监。大部分组长干到第十年，一次都没真正练过。

四 两条路

翻译的价值在往下走，那往哪走？只有两条。

往上：做取舍，担后果

从"把目标翻成任务"，变成"决定做什么、砍什么、先做什么"，然后为结果负责。

难在三处。一是你得懂业务，不知道这事对客户意味着什么，就无从判断哪个更重要。二是你得敢说不，第一次说"这个不该做"最难，对方会不高兴，你还得给出站得住的理由。三是你得承受"错得看得见"——执行错了还能说是需求没讲清楚，判断错了就是你错了，而且所有人都看着。

代价不小，起步却不难：这个季度争取一次砍需求的机会。不用砍大的，砍个中等的，把理由写清楚，然后为结果背书。做完这一次，你就从"拆活的人"变成了"定事的人"。

怎么验证？下次规划会上再有人问该做哪三件，你能答上来，还能说出第四件为什么不做。

往下沉：自己端到端交付

从"带人做"变成"自己能跑完全程"——从理解需求，到上线，到看数据，到挨骂，一个人加 AI 全干下来。

技术上已经不是障碍。前端不会写？AI 写。运维不熟？AI 教。真正的门槛在心理上：你还愿不愿意对最终结果负责。在中间层待久了的人，早就不习惯"这东西是我做的，好坏都算我的"这种状态了。

代价得说实话：从带团队退回自己干，职级和收入预期都要往下调。这不是什么华丽转身，是真的要放弃一些东西。

起步动作：挑一个小项目，端到端自己做完，不派活、不协调、不求人。做完你就知道短板在哪，通常不在技术上，在需求理解和验收标准上。

怎么验证？一个完整的活儿，从想法到用户真的用上，你一个人能交付，而且敢在上面署名。

原地不动是什么样

继续当协调者、推动者、救火队员。你会继续很忙，但你做的事里，AI 能做的比例会越来越高。

然后你会掉进一种很特别的状态：很忙，但没人说得清你的贡献是什么，包括你自己。

这比被裁更危险。裁员是个事件，你会疼，会反应，会去找下家。而这个状态是慢性的，很多人能在里面待三五年，直到某天被一个完全没预料到的决定打个措手不及。

所以判断标准别用"我忙不忙"，用这个：

如果你消失一个月，组织缺的是一种能力，还是少了个传话的？

已经卡住了，第一步怎么迈

上面两条路都要你动起来。现实是很多人已经在中间待了两三年，动不了：手上的活放不下，新的能力还没长出来，中间这段最难熬。

别想着一次转身。先做一件立刻能见效的小事：从下周起，每周挑一件你要派下去的活，自己先想一遍"如果是我定，我砍不砍"，把想法写三行，不用给任何人看。

两个月后你手上会有两样东西：一堆自己的判断记录，和一次能拿出来说的底气。这时候再去争取拍板的机会，把握完全不一样。

还有一件容易忽略的：把你手上的协调活主动让出去一部分。看板、摘要、进度同步，能自动的自动，能轮值的轮值。中间层最难的不是学新东西，是腾出手。

把两条路摆在一起

模 型

把两条路摆在一起

	往上：做取舍的人	往下：自己交付的人
干什么	决定做什么、砍什么	从需求到上线一个人跑完
需要什么	懂业务、敢说不、担后果	端到端能力、肯对结果负责
要付的代价	犯错看得见，没退路	放弃管理身份，收入预期要调
第一步	这季度砍一次需求	独立完成一个小项目
怎么算成了	能说出第四件为什么不做	一个人交付，敢署名

自检三问

1. 如果组里的人全用上 AI，你今天做的哪件事，他们不再需要你做？
2. 过去半年，你做过几次真正的选择？有没有一次是你拍板、后果也算你的？
3. 如果明天团队缩到三个人，你是那个必须留下的人吗？为什么？

可以做的事

个人

- 1 花半小时做一次价值审计：把上周做的事列出来，一项项标——AI 能不能做？做完还有没有人需要？划掉被吃掉的，看看还剩什么
- 2 这季度争取砍一次需求，写清理由，为结果背书
- 3 独立完成一个小项目，不派活、不协调，试试自己能不能沉下去

管理者

- 1 别把组长当传声筒用。把"砍什么、先做什么"的决定权明确下放，同时把要担的后果也说清楚
- 2 考核口径换一换。少看派活效率和进度达成，多看判断质量——砍对了什么，避免了什么损失
- 3 主动找组里那几个"很忙但说不清贡献"的人聊一次，直接告诉他你看到了什么。这种信号他自己往往看不见

别的岗位呢

这章讲的不是"组长"这个职位，是所有中间那层。

项目经理排期是翻译，产品经理把老板的话翻成原型也是翻译，中层把战略翻成 KPI 还是翻译。特征都一样：输入确定、输出能列全、做完能验收。命运也就一样。

行政、财务、法务、咨询里那些"把上面的意思变成下面的动作"的岗位，在同一条曲线上，只是晚个三五年。

不是更努力，是换维度

在泰坦尼克号上努力划桨，不如抬头看一眼冰山。

这一章的三件事

- 面对焦虑最常见的反应是"更努力"，而这是最无效的一种
- "提效不减负"本身就是证据：在执行这个维度上加码，不会让你更安全
- 四个动作，都是在现有工作里换一种做法，不是再往上加量

— 老魏

老魏是组里最拼的人。

来得比谁都早，走得比谁都晚。需求来了从不推，别人不愿意接的活他接，周末群里有人问问题他回。加班排行榜上他永远第一，这事儿组里没人有意见。

工具普及那一年，他是学得最快的那个。别人还在琢磨怎么问，他已经把整套流程跑顺了。他也是组里产出最高的那个——季度统计出来，他一个人顶别人一个半。

一年之后公司调整，名单上有他。

他当时懵了。他来找我，问的第一句话是：

"我不比别人差啊，我比谁都努力，我哪儿错了？"

他没错。他只是把所有的力气，都用在了不需要判断的地方。

回头看他那一年，轨迹是这样的：工具让他变快了 → 于是他接了更多活 → 于是他更忙 → 于是他更没时间想别的 → 于是他做的全是"别人已经定好的事"。

他那个组这一年砍掉的需求有十几个。砍的时候没人问过他，因为他从来不参加评审——他没时间，他在写代码。

他做的东西质量很好，交付很及时，从来不用人催。但如果你问他"你们这块业务明年该往哪走"，他会愣住。

他不是不够努力。他是在一个正在贬值的维度上，把自己加到了极限。

而且这里有个特别残酷的地方：他的努力，让这个循环转得更快。

他越快，标准抬得越快；标准抬得越快，他能用来想事情的时间越少。

二 为什么"更努力"失效了

这不是说努力没用，是说在错误的维度上，努力有反效果。三个原因。

第一，效率增益会被自动吸收。

第1章讲过这个机制。你三天干完一周的活，下次排期就是三天。老魏更快了，于是他接到了更多活，而不是获得了更多空间。

这不是谁在为难他，这是算术。当执行可替代性强的时候，议价能力就弱，效率增益自然归组织。

所以"我做得更多、更快"这条路，通向的不是更安全，是新标准更高。

第二，你加码的那个维度正在贬值。

对照第 2 章那张表：写得快、接得多、交付及时，这几项都在跌的那一侧。

你越是把时间投在跌的那一侧，你的时间就越不值钱。这不是态度问题，是方向问题。

第三，也是最实在的一条：疲惫会挤掉成长的空间。

练判断力、懂业务、做复盘，这些都要精力。而老魏那一年每天回家只想躺着。

这不是意志力问题，是资源约束。一个人的精力是有限的，全部投进执行里，就没有余量投进成长里。

于是出现一个越忙越危险的循环：忙 → 没时间想 → 只能继续做执行 → 执行在贬值 → 更不安全 → 更不敢停 → 更忙。

打破它的办法不是再加一把劲，是从里面跳出来一步。

三 换维度是什么意思

很多人听到"要练判断力、要懂业务"，第一反应是：那我是不是得更忙了？

不是。这本书从头到尾的主张是：不是在你现在的活上加量，是在同一件活上换一种做法。

举几个例子，你就明白差别在哪：

模 型

换维度是什么意思

原来的做法	换一种做法	多花的时间
接到需求就开始做	先花十分钟写三个答案：为什么做、为谁做、做成什么样算好	十分钟
做完就交	上线后看一眼数据，看谁在用、怎么用	二十分钟
需求来了就排期	评审时问一句"不做会怎样"	一句话
工具生成的代码改改就提交	先想清楚这块要验收什么，再让它写	五分钟
决定做完就算了	做之前先写下"什么情况说明我错了"	三行字

这些都不是加班能解决的，也不是加班能替代的。它们跟"做多少"无关，跟"怎么做"有关。

老魏那一年的产出，如果拿出十分之一的时间来做右边这栏，结果会完全不一样。

而且注意一件事：右边这栏的很多动作，实际上会减少你的工作量——"不做会怎样"这一句问对了，可能砍掉两周的活。

所以换维度不只是更有效，它还可能更轻松。前提是你得先停下来那十分钟。

这也是序里我写那句话的原因：这本书给的动作都尽量压到最小，因为读者大多已经很累了。

但我也也不想骗人：开头那一点点是必须挤的。关键不在加多少，在加在哪个维度上。

四 四个动作

说具体点。想往"研"那一侧挪，四个动作就够，而且都不需要额外的时间块。

第一，追问三个为什么。

接到任何一件事，先问：为什么要做、为谁做、不做会怎样。

三个问题问完，你大概能判断出这个需求是真需求，还是某人随口一提被记了下来。

大部分人从不问，因为问了显得事儿多。但你问过一次就会发现，对方的回答往往会让他自己也重新想一遍。

第二，去现场。

见真实用户、看工单、翻客服记录、跟着销售跑一趟。

第 5 章的小许，四年不如那两周。就是因为业务理解这东西不在文档里，在现场。

成本是多少？一个月半天。

第三，看结果。

你做的东西上线之后，真实数据怎么样、用户怎么骂的、有没有人真的在用。

这一条最容易被忽略，也最容易做到。因为大部分研发做完就走，从来不看。

看一眼，你就会开始想"下次该怎么做"，这就是"研"。

第四，练习拒绝。

试着说一次"这个不该做"，并且给出理由。

这是检验业务理解最好的试金石——说不出来，说明你还在 L2 以下（第 5 章）。说得出来，你就有了一次做判断的练习。

第一次最难。可以先从小事开始，比如砍掉一个明显没人用的小需求。

五 不同处境，重心不一样

如果你是刚入行的应届生。

先给一条反直觉的建议：别急着学工具。

工具会变，而且门槛只会越来越低。你现在花三个月练熟的某个东西，两年后可能一句话就能搞定。

你现在该攒的是"判断的素材"——见过足够多的"决策—结果"对。

具体讲：在学校或者刚工作的头两年，做 3 到 5 个完整交付给真实用户的项目。关键是"完整"和"真实用户"：不是课程作业，是有人真的在用，用完了会抱怨。

比刷一百道算法题有价值得多。因为前者给你的是判断样本，后者给你的是手速，而手速正在贬值（第 1 章那张表）。

另外，第 9 章那条赛道（驾驭者）是专门给年轻人留的，认真看。

如果你工作了三到五年。

这是最关键的窗口期，因为你还便宜，试错成本低。

重心放在两件事：主动接要拍板的活（第 6 章），和搞懂你做的东西给谁用（第 5 章）。

这两件事都不需要换工作、也不需要加班，只需要在现有工作里伸手。

这个阶段的人最容易犯的错是"再学一门语言、再熟一个框架"。那是往括号外面加东西（第 4 章），收益很小。

如果你干了十年以上，正在焦虑。

别去补手速，那是别人的主场。也别急着转管理——如果只是想逃避技术，那会更难受。

你的重心是：把已有的经验挂到业务上。

你已经有技术判断了，缺的是"这个决定对客户、对收入意味着什么"。补上这一块，你手里那十几年的东西才会重新值钱——老郑那个故事（第 2 章）就是这个意思，他攒的东西没变，是卖法变了。

另外找那个"只有你能定"的位置。这个年纪再去比手速没有意义，你的价值在于别人拿不准的时候，你能拿得准。

如果你想往管理走。

先想清楚一件事：管理的第一优先级不是管人，是带领业务发展。下一章整章讲这件事。

六 时间从哪来

最后一个很实际的问题：我已经很忙了，这些动作塞在哪？

三个办法。

第一，嵌进现有动作里。上面那张表的右栏，大部分是几分钟的事，不需要单独的时间块。接需求的时候多问一句，做完之后多看一眼，就这些。

第二，用工具换时间，但别把换来的时间全投回执行。这一条最重要。工具让你省了两小时，别马上接两个新活，留半小时给自己。

第三，学会说"这个我做不了"。老魏那一年的教训就是从不拒绝。你接下的每一件不该做的事，都在挤掉你成长的空间。

说到底一句话：不是让你做更多，是让你在做的事里，多想一步。

而那一步，正是工具永远替你走的那一步。

自检三问

- 1 过去半年，你加班变多了还是变少了？如果是变多了，你换来的东西变多了吗？
- 2 你现在的忙碌里，有多大比例是"别人已经定好了，你负责实现"？
- 3 如果每天给你一小时，你知道该用来做什么吗？

可以做的事

个人

- 1 这周问一次"为什么要做、为谁做、不做会怎样"。就一次
- 2 去看一眼你半年前做的东西现在的使用数据。看完你会有想法
- 3 挑一件小事说"不"。先从小事开始，别一上来挑大的
- 4 工具给你省下的时间，这周留半小时给自己，别马上接新活

管理者

- 1 看一眼团队里谁最忙。最忙的那个人，往往是最没时间成长的人——这不健康
- 2 别表扬加班。你表扬什么，团队就给你什么
- 3 给"想事情"留合法时间。哪怕只是评审会前那十分钟
- 4 检查一下：团队里有没有人被"能干"这个标签困住了，因为他能干，所以活都给他，于是他永远没机会做别的

别的岗位呢

"更努力"这个陷阱不是研发专属。

设计师用工具出图更快了，于是接更多稿，于是更累，于是更没时间研究用户。分析师做报表更快了，于是做更多报表，于是更没时间想"这些报表影响了什么决策"。

规律一样：工具让你在"做"这个维度上更强，而"做"正是最容易被吸收掉的一层。

越努力，越忙，越没时间抬头。这个循环跟岗位无关，跟维度有关。

组织这一侧

从"管人"到"带领业务发展"

管理的第一优先级不是管人，是带领业务发展。

这一章的三件事

- "管人"本质上也是翻译和协调，正在被工具吃掉；"带领业务"是判断，工具做不了
 - 组织该改的三件事：下放决策权、绩效从产出量转向判断质量、把信息透明化
 - 顺便回答那个很普遍的问题：总觉得领导在压榨我，到底是不是
- ### — 老陈的后来

第 10 章讲过老陈。季度规划会上那句"得看产品那边怎么排吧"，说完没人接话，投屏上的光标一直在闪。

后来他换了个 leader。

新 leader 上任第二个月，找他谈了一次。谈的内容不是"你要多承担"，是把一件事交给他：下季度你们组做什么、砍什么，你定。

同时说了后半句：定了就得担，砍错了也算你的，但我会替你兜住外面的压力。

老陈后来跟我说，那次谈话他紧张了两周。

因为前四年他从来没做过这种事。第一次真的要砍的时候，他挑了个中等的：一个老客户提的导出功能，一年用不了两回。他写了半页纸的说明：谁提的、现在怎么解决、做了要多久、不做会怎样。

发出去之前他改了三遍。发完之后他盯了一个星期，看有没有人反对。

没有。

第三个月，他在评审会上说了第二次。这次他砍了个大一点的，理由是那批客户里有一半已经在用别的方案了。有人问了两句，他答上来了。

半年之后他们组的状态变了。最明显的变化不是产出，是开始有人在评审会上说"这个不该做"了——先是老陈，然后是组里两个年轻人。

以前没人说，因为说了也没用，还可能得罪人。现在说了有用，是因为 leader 真的会听。

老陈那个 leader 后来跟我讲过一句挺朴素的话：

"我也没做什么，就是把'你定'这两个字说出来了，然后真的让他定。"

把决定权给出去，看着简单，是最难的一步。因为你要忍住不插手，还要在别人质疑的时候站在他前面。

二 "管人"正在被吃掉

我们认真想一下：一个管理者平时在做什么？

- 把目标翻成任务
- 把任务分下去
- 盯进度、拉对齐
- 上传下达
- 处理冲突、救火
- 向上汇报

这六项里，前四项是什么？

翻译和协调。

把目标翻成任务是翻译（第 10 章讲过，拆任务就是一种翻译）。盯进度、上传下达，本质是传递信息——你是那根管子。

而这四样，正是工具接得最快的那部分。任务拆解工具比人细，进度看板自动出摘要，汇报材料系统自己能生成。

所以中间层被掏空（第 10 章）不是意外，是这四样被接走的必然结果。

那剩下什么？

判断：这块业务该往哪走、做什么不做什么、什么算做得好。

这一样工具做不了，因为它需要对客户、对市场、对组织处境的理解，需要承担后果。

所以管理的重心必须挪，从“管人”挪到“带领业务发展”。

公司真正需要的，不是能管住一群人的人，是懂业务、并且能带着一群人把业务做得更大的人。

这也是这本书一开始那个名字的意思——「研发领航」的"领航"，不是管人，是给业务定方向。

一条船上的领航员，职责不是让船员多划几下，是知道该往哪走、哪有礁石、什么时候该改道。

三 组织该改的三件事

管理者能动的杠杆比个人大。三件事，按重要性排。

第一，把决策权下放

这一条在第 6 章讲过，这里说组织层面。

下放的判据很简单：错了能不能回滚。能回滚的决定，就该放下去。

大部分团队不敢放，理由通常是"放下去会出事"。但你算一笔账：放下去出的那些事，损失的往往是几周；不放下去，损失的是整个团队十年不会做决定。

后者更贵，只是它不体现在任何一张报表上。

下放的时候还要注意一件事：权力要和责任一起给，但责任不能给得比权力多。

很多组织的做法是"这事你负责"，但真到要资源、要支持的时候，还是得层层上报。这不叫下放，叫甩锅。

正确的说法是那两句话：你定；定了算你的，但外面的压力我替你兜。

第二，绩效从"产出量"转向"判断质量"

第 2 章讲过：能数的都在跌，涨的那些不好数。

代码行数、任务数、按时交付率，这些最好统计，也贬值最快。而"砍掉了一个不该做的需求""提前判断出这个方案会出事"这类贡献，几乎不可能进考核表，因为它是"没发生的事"。

但这个困难不是不改的理由。几个能落地的办法：

- 考核里加一条"砍对了什么"，哪怕只是口头认可
- 看上线之后的使用情况，不看交付个数。这一条数据自己会说话
- 复盘会上表扬"提前发现风险"的人，而不是只表扬救火的人
- 晋升答辩问"哪件事是你定的"，不问"你参与了多少项目"

还有一条比较难但很关键：别表扬加班。

你表扬什么，团队就给你什么。你表扬加班，就会得到一群疲惫的、没时间成长的人。

第三，把信息透明化

这一条看起来最温和，其实是解决信任问题最有效的手段，下一节展开。

四 看你把时间花在哪

有个很简单的自检：翻一下你上周的日历，看时间都花在什么上。

如果大部分是"过需求、对进度、写汇报、救火"，那你做的是协调，而协调正在被接管。如果大部分是"跟客户聊、决定做什么不做什么、跟人谈他要承担什么"，那你做的是带领业务。

中间那段怎么挪？四个具体动作：

- 把进度会从你主持改成看板自己说话。你主持进度会，是因为信息不对称；看板加自动摘要之后，这件事不需要你了。省下来的时间拿去见客户
- 每周留半天不给任何人。这半天用来想"下个季度我们该做什么"，不是处理最急的那件事。很多 leader 的日程里从来没有这种时间块，因为所有空隙都会被会填满了
- 把"该不该做"在会上问出来，然后等人回答。别自己先给答案。你一开口，下面就变成了执行讨论
- 每季度跟团队讲一次我们在扛什么。讲指标、讲约束、讲上头在催什么。讲完你会发现，很多抱怨不是因为活多，是因为不知道为什么

这四件事的共同点是：都在减少你"传递信息"的时间，增加你"做判断"和"让信息透明"的时间。

五 关于"领导是不是在压榨我"

这个问题在第一章之后一直悬着，这里正面回答。

先说结论：这种感受不全是误解，确实有压榨；但更常见的情况，是两边看到的信息不一样。

为什么很多执行者会把所有额外要求理解为压榨？

根源在定位。

一个只把自己定位成执行者的人，心里的投入产出是线性的——多干就是多吃亏。你让他多干一小时，他就觉得自己亏了一小时。在这个框架下，任何额外要求都只能被理解为压榨。

这不是人品问题，是位置决定的视角。站在执行的位置上，你看到的是"我的付出"和"我的回报"；你不知道这个需求为什么突然插进来，不知道上头在扛什么指标，不知道这个季度公司账上是什么情况。

所以破解的办法不是沟通技巧，不是"多换位思考"。

是让他进入判断链条。

具体讲就是三件事：

1. 参与决策。让他知道为什么做这个、为什么是现在，而不是只拿到一个任务
2. 了解决策背后的处境。团队在扛什么指标、公司面对什么压力、有哪些约束。这些信息大部分管理者习惯性地不往下讲
3. 给他一部分决定权。一旦他自己做过取舍，他就会明白"砍掉一个需求"有多难，也就会理解为什么有些要求看起来不合理

老陈那个组半年后的变化就是这个道理。当组里的人开始参与"做什么不做什么"的决定，怨气自然就少了：不是因为活变少了，是因为他们知道了为什么。

一个人从执行者变成参与者，怨气会自己消解。

也要诚实地说另一面：确实存在真实的压榨，有些管理者就是把人当耗材。这种情况这本书帮不了你，该走就走。

还要说一句我自己也没做到位的话：上面那三条，我在自己团队里也就做到两条半。讲道理容易，每周真把处境讲清楚，很难坚持。

但更多时候的真实情况是：你的 leader 也在被卷，只是你没看见。他也在被要求更少的投入、更多的产出，他也在会上被问"为什么你们组不能更快"，他也不敢说"做不完"。

第 2 章那个画面——"数据上这个组更高效了，里面的人却更累、更焦虑，而且不敢说"——从 leader 到一线，是一条链上的人都在里面。

这不是为谁开脱，是说清楚一件事：把问题归结为"领导坏"，除了让自己更难受，不解决任何问题。

六 还有四件管理者容易忽略的事

第一，效率增益别全额收走。

短期看，把工具带来的提效全部变成产出，是最划算的。长期看，一个全员疲惫且不敢说话的团队，迟早会在某个没人预料到的地方出事——要么是质量事故，要么是一次集体流失。

留出一部分还给团队，哪怕只是"少接一点需求"。这笔账在一个季度内看不出来，两年后看得出来。

第二，真要减员，想清楚按什么顺序。

只按价格和年龄砍，砍掉的不只是成本，还有稳定性。留下的每个人都会开始算"下一个是不是我"，而这个念头比加班更消耗人。

而且砍错了顺序会砍掉隐性知识：那些没写进文档、存在某个人脑子里的东西（第 2 章老郑那条）。这类损失不会立刻显现，半年后才会以"怎么又出这种事"的形式回来。

第三，人少了，要求得跟着变。

团队规模可以缩小，但对每个人的要求要提高。一个五人团队，如果五个人都只会做定好的活，不如三个人但每个人都能独立扛一件事。

还有一点：别把所有能干的人堆在一个组。老魏那种“最能干”的人（第 11 章），如果组里有三四个反而是灾难，因为所有人都在等他，而他没有时间成长。

第四，别先减架构和资深。

提效之后算成本，最先被盯上的通常是两种人：贵的，和不直接写需求的。架构师、做规范和治理的人，正好两样都占。

短期看这笔账很划算，报表立刻好看。长期看，你要的东西会以另一种形式加倍还回来：产出越多，系统的一致性越脆，而守住一致性的正是这批人。

同一件事，三个小组用工具各写各的，每块单看都没问题，半年后拼不起来。返工的成本比省下的人头费高得多（第 6 章第八节讲过这个机理）。

工具用得越多，越需要有人守住边界、依赖和命名。这块人减了，不会立刻出问题，等出来就是结构性的。

自检三问

- 1 你手里握着多少决定权？其中有多少是“错了能回滚”、完全可以放下去的？
- 2 你的团队里，有人说过“这个不该做”吗？上一次是什么时候？
- 3 你的团队知道你们在扛什么指标、面对什么约束吗？

可以做的事

个人

- 1 如果你觉得领导在压榨你，先问自己一个问题：我知道这个季度团队在扛什么吗？不知道的话，去问一次
- 2 主动争取一次参与决策的机会，哪怕只是列席一次规划会
- 3 试着用自己的位置想一次：如果你是 leader，这个季度你会怎么排？

管理者

- 1 列一张清单：你手里的决定里，哪些是错了能回滚的？挑三个，这个季度放下去
- 2 跟团队讲一次你们在扛什么、面对什么约束。一次就够，效果会超出你的预期
- 3 考核里加一条"判断质量"，哪怕只是口头表扬。你表扬什么，团队就长什么
- 4 减员之前想清楚顺序，并且把"按什么标准"讲清楚。模糊比坏消息更伤人
- 5 效率增益留一部分还给团队。这是长期账

别的岗位呢

这一章讲的不只是研发团队的管理。

职能部门、销售团队、内容团队——凡是"把上面的意思变成下面的动作"的中间层，都在同一条曲线上：翻译和协调被接走，剩下的只有判断。

而破解"被压榨"的方式也一样：让执行者进入判断链条。这在任何岗位上都成立，因为怨气的根源从来不是活多，是不知道为什么。

未来在哪

这本书从一个问题开始：接下来怎么办。

十二章写下来，我想把答案收成一句话。

AI 放大的是你已有的东西。所以先让自己有东西可被放大。

这句话有两个方向。往坏的方向看：如果括号里几乎是空的：不判断、不懂业务、说不清楚——那么工具再强，放大的还是一个接近零的数。往好的方向看：同样一份判断力，现在能撬动以前十倍的产出。

所以这一轮真正的分化，不是"会用工具的人"和"不会用的人"。工具很快会变成默认项，人人都会。真正的分化是：括号里有东西的人，和没有的人。

给刚入行的你

你可能正在焦虑：现在工具这么强，我学的东西还有用吗？

有用的，但有用的部分跟你以为的不太一样。

有用的不是手速，不是记住多少 API，是你做过的那些完整的事。所以我给你的建议一直没变：别急着囤工具，先做三到五个完整交付给真实用户的项目。有人真的在用，用完了会抱怨，你得去处理。

这些经历会变成你的判断样本，而这些样本是唯一不能被速成的东西。

另外记住第9章那条赛道。它是这几年新长出来的，在座所有人都没有五年以上的经验。这是你唯一一条不吃资历亏的路，别浪费它。

还有一件事：不要怕入行晚、不要怕起点低。这一轮里，决定你位置的不是你干了多少年，是你有没有在做判断。而做判断这件事，从第一天就可以开始。

给干了十年、正在焦虑的你

你可能正在经历那种很特别的疲惫：活更多、人更少、自己更快，但处境一点没变好。而且你不敢说，因为"效率上来了"已经定下了调子。

先说一句：这份疲惫是真实的，不是你矫情。它是这一轮变化的必然结果，不是你的问题。

然后说一句可能不太好听的：别再往手速上加码了。

我见过太多人在焦虑之下做的第一件事是"再把技术捡一捡，学点新的"。这条路的尽头，是跟二十来岁的人比谁学得快。你赢不了这个，也不该去赢。

你该做的是把已有的东西挂到业务上。

你这十几年攒的东西没变，变的是卖法。以前卖的是"我能把定好的事做好"，以后要卖的是"这块东西交给我，你不用管了"。老郑那句话说的就是这个（第2章）。

具体怎么转？三件事：搞懂你做的东西给谁用、主动接一次要拍板的活、试着说一次"这个不该做"。都不需要换工作，也不需要加班。

最后一句：你的经验不是负债，是没被激活的资产。激活它只需要一个动作——从"别人决定，我实现"，走到"我决定，我负责"。

给带团队的人

你的第一优先级不是管人，是带领业务发展。

这句话要反复说，因为太多人把管理理解成了“把人管住、把活分下去、把进度盯紧”。这三件事都是翻译和协调，都在被工具接走。

留下来的是判断：这块业务该往哪走、做什么不做什么、什么算做得好。而这些判断，工具做不了。

所以你要做的三件事很朴素：

把决定权放下去。错了能回滚的，就该放。你忍住不插手的那几次，就是团队长本事的几次。

把考核改一改。少看产出量，多看判断质量。你表扬什么，团队就长什么。

把信息讲清楚。团队在扛什么、面对什么约束、为什么现在要这么做。讲一次，怨气能消掉一大半。

还有一个提醒：别把效率增益全额收走。一个全员疲惫且不敢说话的团队，迟早会在某个没人预料到的地方出事。这笔账短期看不出来，两年后看得出来。

以及最后一件——留意你团队里那些“很忙但说不清贡献”的人。他们往往是最努力的，也是最危险的。主动找他们谈一次，把决定权给一点。这种信号他们自己看不见。

最后

回到这本书一开始那个名字——「研发领航」。

这也是我对“未来在哪”这件事最终的回答：

未来不在更快的划桨上，在判断方向上。

不安是真实的。岗位在收缩，年龄偏见存在，很多人确实很累。这本书没有粉饰这些，也不打算说"只要你努力就不会被淘汰"。

但有一件事我确定：这一轮里，能被工具替代的，是"把定好的事做好"；不能被替代的，是"知道该做什么、为什么做、做成什么样算好"。

前者是发，后者是研。

过去几十年，我们这行把重心放在了"发"上，因为翻译很贵。现在翻译便宜了，重心该挪了。

从实现者到定义者。

这条路不轻松，但它是通的。而且现在开始，还来得及。

我在半路上，我们同一条船。